ARTIFICIAL INTELLIGENCE

Lecture Set 08
Adversarial Search

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
7. Constraint Satisfaction Problems
8. **Adversarial Search**
9. Logical Agents
10. First-order Logic
11. Uncertain Knowledge and Reasoning
12. Supervised, Unsupervised, and Reinforcement Learning
13. Recent Trends in AI and Applications of AI Algorithms

Table of Contents

Multi-agent Environment

- Each agent needs to consider the actions of other agents and how they affect it's own welfare.
- The unpredictability of these other agents can introduce contingencies into the agent's problem-solving process.
- Two types:
  - *Competitive* : When agent B is trying to do something that maximizes its performance measure and minimizes other agent A's performance measure.

    For example, playing chess.
  - *Cooperative*: When both agents are trying to do something that maximizes both agents performance measures.

    For example, taxi-driving agents.

Adversarial Search

- Adversarial Search
  - In this search problem, agents' goals are in conflict creating a competitive environment.
  - It is a search when there is an enemy or opponent changing the state of the problem every step in a direction you do not want.
  - Examples,
    Chess, Tic-tac-toe, 4 in a row, Checkers, Go, Othello etc.

- AI Game Theory
  - Deterministic
  - Turn taking
  - Two player
  - Zero-sum game = It describes a situation in which a participant's gains or losses is exactly balanced by the losses or gains of the other.
  - Perfect information
  - Agents are restricted to a small number of actions described by rules

Game >> Formal Definition

A game can be formally defined as a kind of search problem with the following elements:

- S_0 : The initial state, which specifies how the game is set up at the start.
- *PLAYER(s)* : Defines which player has the move in a state.
- *ACTIONS(s)* : Returns the set of legal moves in a state.
- *RESULT(s, a)* : The transition model, which defines the result of a move.
- *TERMINAL-TEST(s)* : A terminal test which is true when the game is over and false otherwise. States where the game has ended are called terminal states.
- *UTILITY(s, p)* : A utility function that defines the final numeric value for a game that ends in terminal state *s* for a player *p*.

Game Tree >> The initial state, ACTIONS function, and RESULT function define the game tree for the game. It is a tree where the nodes are game states and the edges are moves.

Our target is to find the Optimal Strategy … …

Types of Games in AI

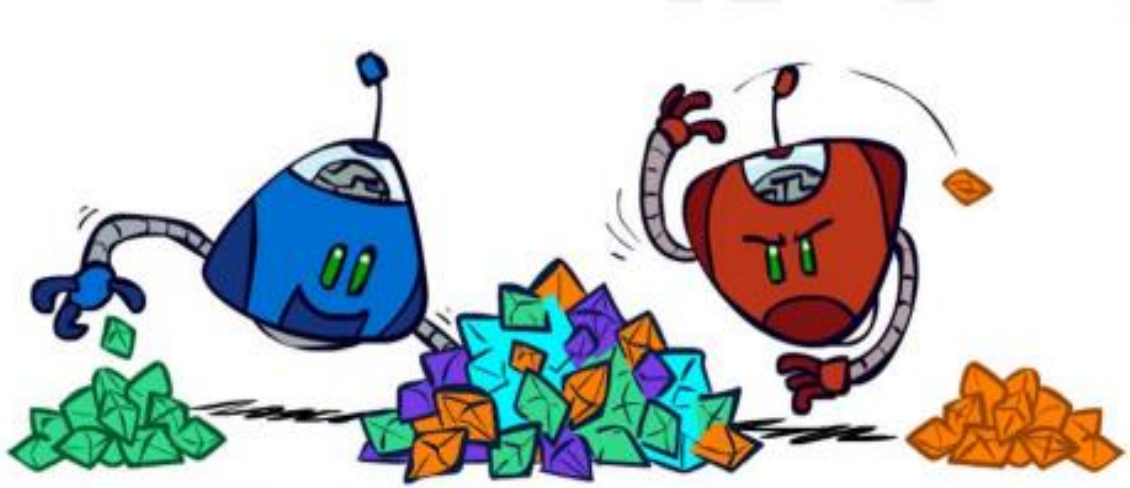
Games are modeled as a Search problem and evaluation function, that helps to solve games.

	Deterministic	Non-deterministic
Perfect information	Chess,	Backgammon, monopoly
Imperfect information	Battleships	poker, scrabble

Perfect information: A game with the perfect information is that in which agents can look into the complete board.
Imperfect information: Agents do not have all information about the game
Deterministic games: Deterministic games are those games which follow a strict pattern and set of rules for the games.
Non-deterministic games: Non-deterministic are those games which have various unpredictable events

Types of Games in AI

- Zero-Sum Games
  - Agents have opposite utilities (values on outcomes)
  - Lets us think of a single value that one maximizes and the other minimizes
  - Adversarial, pure competition



- Non Zero-Sum Games
  - Agents have independent utilities (values on outcomes)
  - Cooperation, indifference, competition, and more are all possible

Zero-Sum Game

Zero-sum games are adversarial search which involves pure competition.
In Zero-sum game each agent's gain or loss of utility is exactly balanced by the losses or gains of utility of another agent.
One player of the game try to maximize one single value, while other player tries to minimize it.

General games

Agents have independent utilities
Co-operative, compete or indifference

Game >> Key Properties

1. **Two players alternate moves**
2. **Zero-sum: one player's loss is another's gain**
3. **Clear set of legal moves**
4. **Well-defined outcomes (e.g. win, lose, draw)**

- **Examples:**

  Chess, Checkers, Go,

  Mancala, Tic-Tac-Toe, Othello …

Characteristics of games

- All the utility is at the end state

- Different players in control at different states

Formalizing the Game setup

1. **Two players: *MAX* and *MIN; MAX* moves first.**
2. ***MAX* and *MIN* take turns until the game is over.**
3. **Winner gets award, loser gets penalty.**

- **Games as *search*:**

  Initial state: e.g. board configuration of chess

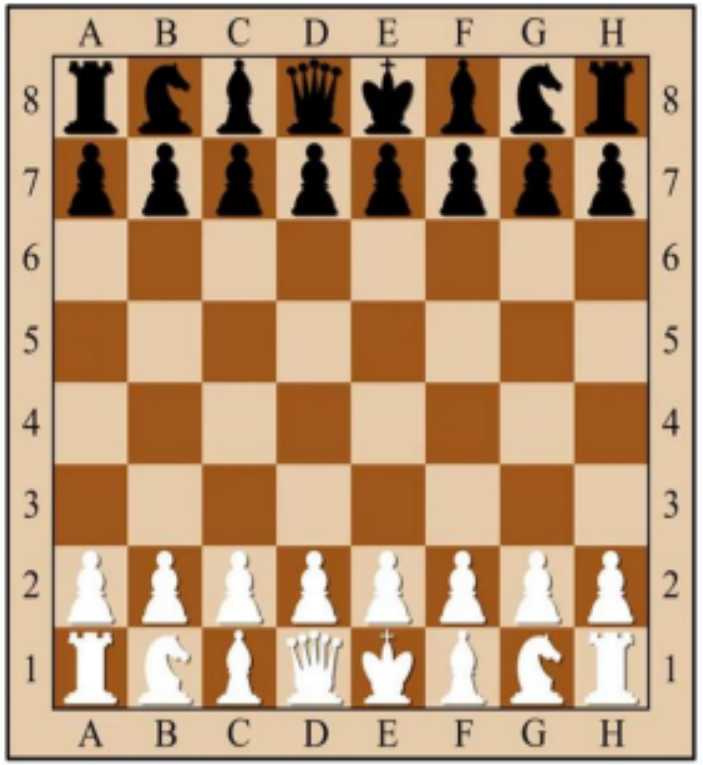
  Successor function: list of (move,state) pairs specifying legal moves.

  Terminal test: Is the game finished?

  Utility function: Gives numerical value of terminal states.
  e.g. win (+∞), lose (-∞) and draw (0)

  MAX uses search tree to determine next move.

Example: Chess



- Players = {white, black}
- State(s): (position of all pieces, whose turn it is)
- Actions(s): chess pieces that Player(s) can move
- Succ(s): the resulting legal moves after a piece ‘a’ is selected.
- IsEnd(s): whether s is a checkmate or draw condition
- Utility(s): +∞ if white wins, 0 if draw, -∞ if black wins

How to Play a Game by Searching

- **General Scheme**
  1. **Consider all legal successors to the current state ('board position')**
  2. **Evaluate each successor board position**
  3. **Pick the move which leads to the best board position.**
  4. **After your opponent moves,** repeat.

- **Design issues**
  1. Representing the 'board'
  2. Representing legal next boards
  3. Evaluating positions
  4. Looking ahead

MAX & MIN Nodes : An egocentric view

- Two players: MAX, MAX's opponent MIN
- *All play is computed from MAX's vantage point.*
- When MAX moves, MAX attempts to MAXimize MAX's outcome.
- When MAX's opponent moves, they attempt to MINimize MAX's outcome.
  - WE TYPICALLY ASSUME MAX MOVES FIRST:

- Label the root (level 0) MAX
- Alternate MAX/MIN labels at each successive tree level *(ply).*
- *Even levels* represent turns for MAX
- *Odd levels* represent turns for MIN

Game Trees

- **Represent the game problem space by a tree:**
  Nodes represent 'board positions'; edges represent legal moves.
  Root node is the first position in which a decision must be made.

- **Evaluation function *f* assigns real-number scores to `board positions' *without reference to path***
- **Terminal nodes represent ways the game could end, labeled with the desirability of that ending (e.g. win/lose/draw or a numerical score)**

A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players. Game tree involves initial state, actions function, and result Function (utility function).

Evaluation functions: *f(n)*

- **Evaluates how good a 'board position' is**
- **Based on *static features* of that board alone**
- **Zero-sum assumption lets us use one function to describe goodness for both players.**

  f(n)>0 if MAX is winning in position *n*

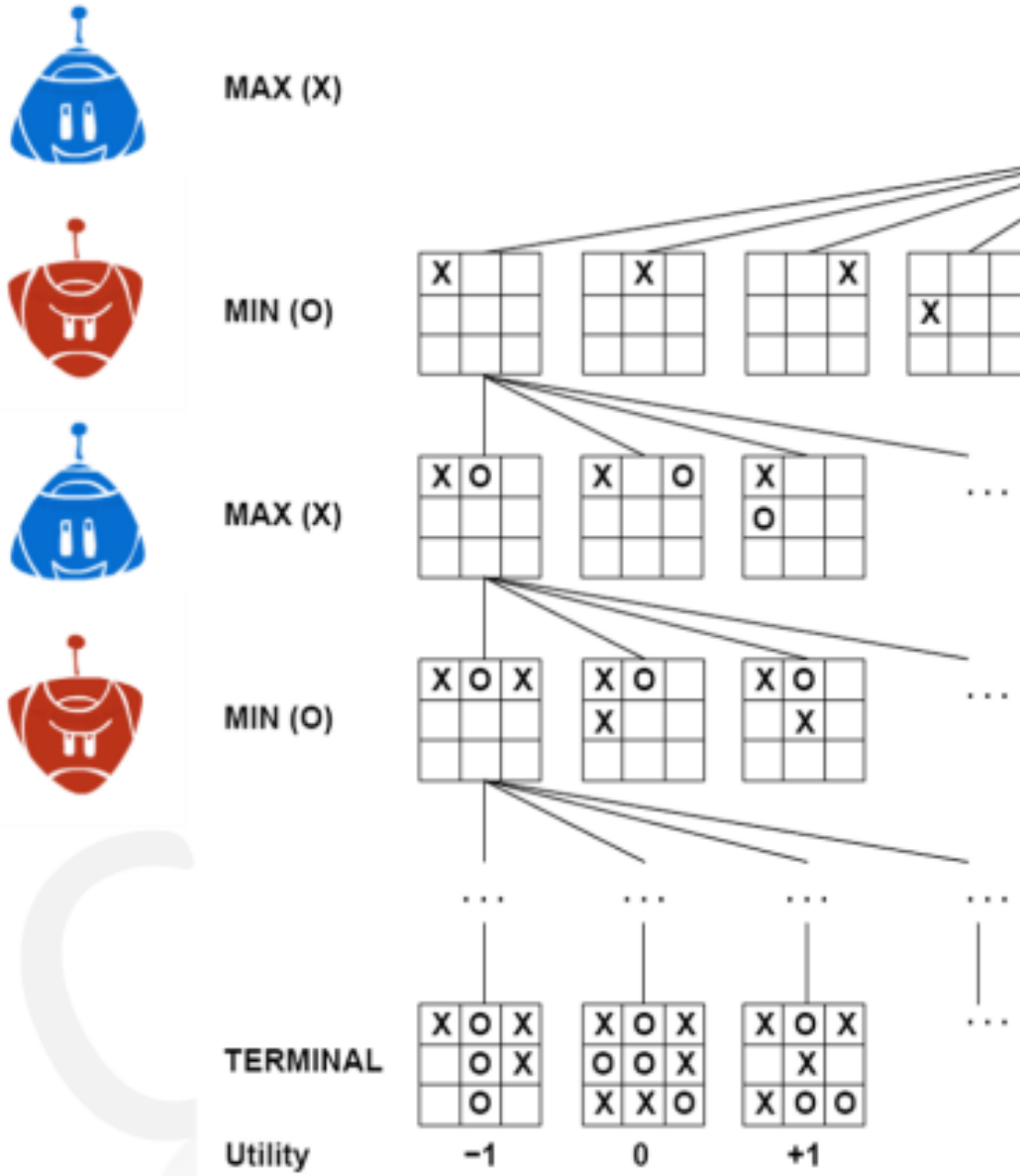
  f(n)=0 if position *n* is tied

  f(n)<0 if MIN is winning in position *n*

- **Build using expert knowledge,**

  Tic-tac-toe***: f(n)=(# of 3 lengths open for MAX)- (# open for MIN)***

Tic-Tac-Toe Game Tree



- Root node represents the initial board configuration; MAX must decide the best single move to make next
- Children represent the possible legal moves for a player
- Each level of the tree has nodes that are all MAX or all MIN
- A utility function determines the value of a terminal node (e.g., +1 for win, -1 for loose, ….)

Evaluation function

- **Utility values of terminal nodes are determined based on a heuristic evaluation function**
  - An evaluation function returns an *estimate* of the expected utility of the game from a given position, just as the heuristic functions in search algorithms return an estimate of the distance to (or cost of reaching) the goal.
- Zero-sum assumption lets us use a single evaluation function to describe goodness of a board w.r.t both players
- Function E: state s $\rightarrow$ number E(s)
  - E(s) is a heuristics: estimates how favorable s is for MAX
  - E(s) > 0 → s is favorable to MAX (the larger the better)
  - E(s) < 0 → s is favorable to MIN
  - E(s) = 0 → s is neutral

- Why using backed-up values?
  - At each non-leaf node *n*, the backed-up value is the value of the best state that MAX can reach at depth *h* if MIN plays well (by the same criterion as MAX applies to itself)
  - If E is to be trusted in the first place, then the backed-up value is a better estimate of how favorable STATE(*n*) is than E(STATE(*n*))

Game Complexity

State-space complexity:

- number of legal game positions reachable from the initial position of the game
- an upper bound can often be computed by including illegal positions
- For example,

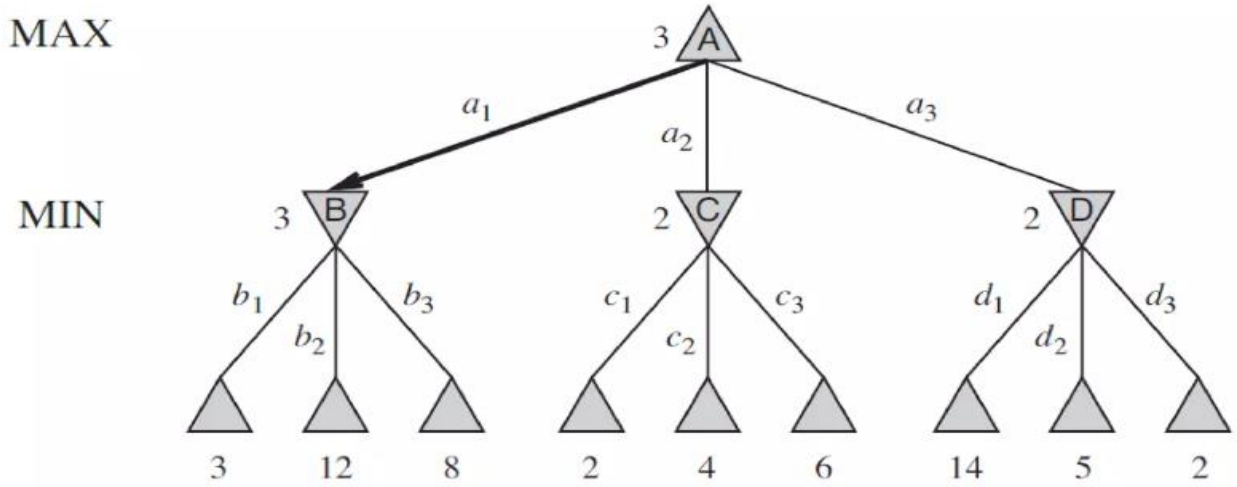
TicTacToe has $3^9 = 19683$ game states (legal + illegal)

Game tree size:

- total number of possible games that can be played
- number of leaf nodes in the game tree rooted at the game's initial position
- For example,

TicTacToe game tree has 9! = 362880 possible games.

Even a simple game like tic-tac-toe is too complex for us to draw the entire game tree on one page, so we will switch to the trivial game as shown below:



The Minimax Rule (AIMA 5.2)

The Minimax Rule: “Don’t play hope chess”

- *Idea*: Make the best move for MAX *assuming that MIN always replies with the best move for MIN*

- **Easily computed by a recursive process**
  - The ***backed-up value*** of each node in the tree is determined by the values of its children:

  - For a **MAX** node, the backed-up value is the ***maximum*** of the values of its children *(i.e. the best for MAX)*

  - For a **MIN** node, the backed-up value is the ***minimum*** of the values of its children *(i.e. the best for MIN)*

Min-Max Algorithm

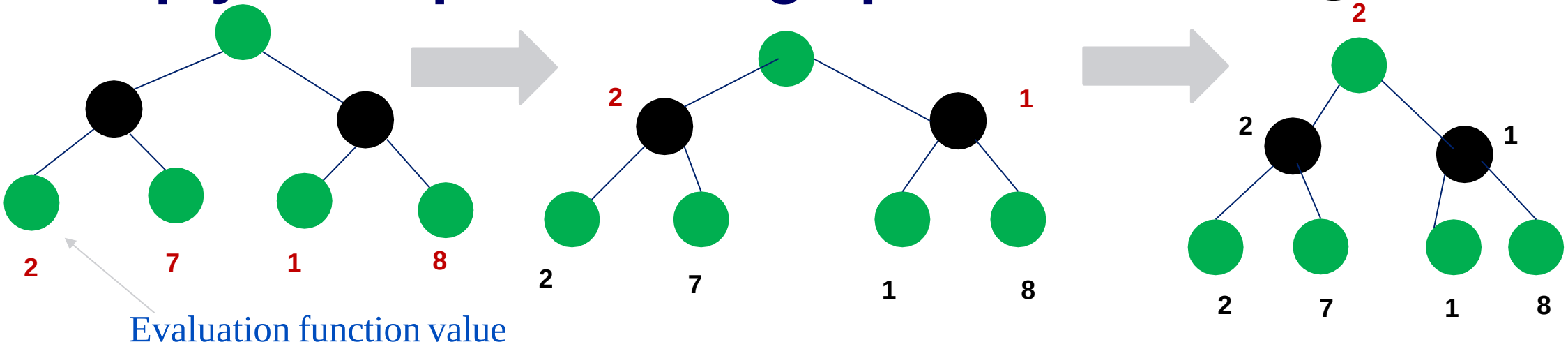
In this algorithm two players play the game, one is called MAX and other is called MIN.
Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
The min-max algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
The min-max algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

The Minimax Procedure

- **Until game is over:**

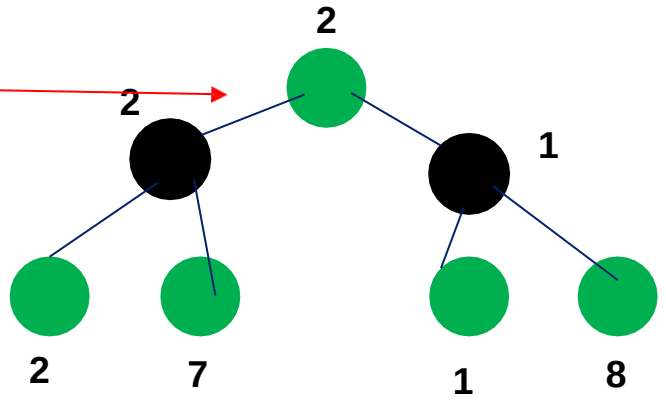
1. **Start with the current position as a MAX node.**
2. **Expand the game tree a fixed number of *ply*.**
3. **Apply the evaluation function to the leaf positions.**
4. **Calculate back-up values bottom-up.**
5. **Pick the move assigned to MAX at the root**
6. **Wait for MIN to respond**

2-ply Example: Backing up values



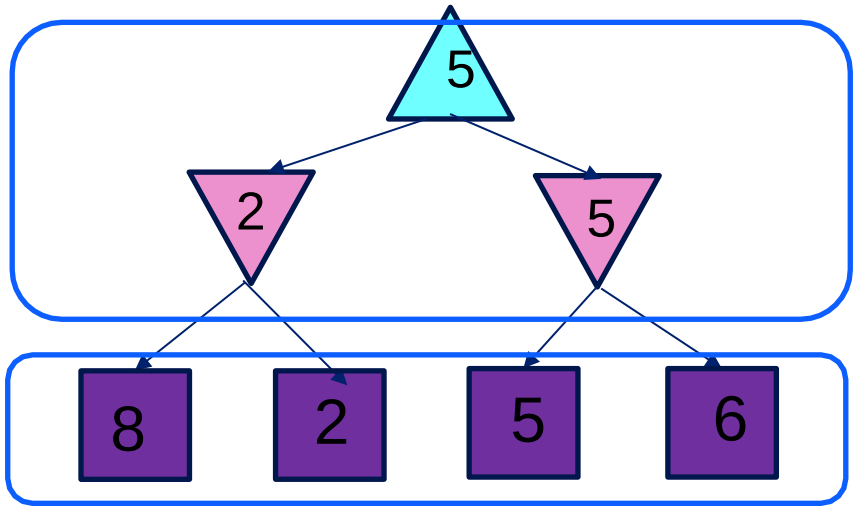
Evaluation function value

This is the move
selected by minimax



Adversarial Search (Minimax)

- **Minimax search:**
  - A state-space search tree
  - Players alternate turns
  - Compute each node's minimax value: the best achievable utility against a rational (optimal) adversary



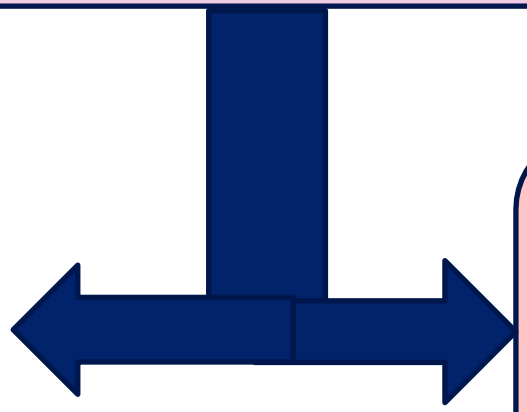
Minimax Implementation

```
def max-value(state):
    if the state is a terminal state:
        return the state's utility
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```

```
def min-value(state):
    if the state is a terminal state:
        return the state's utility
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

Minimax Implementation (Dispatch)

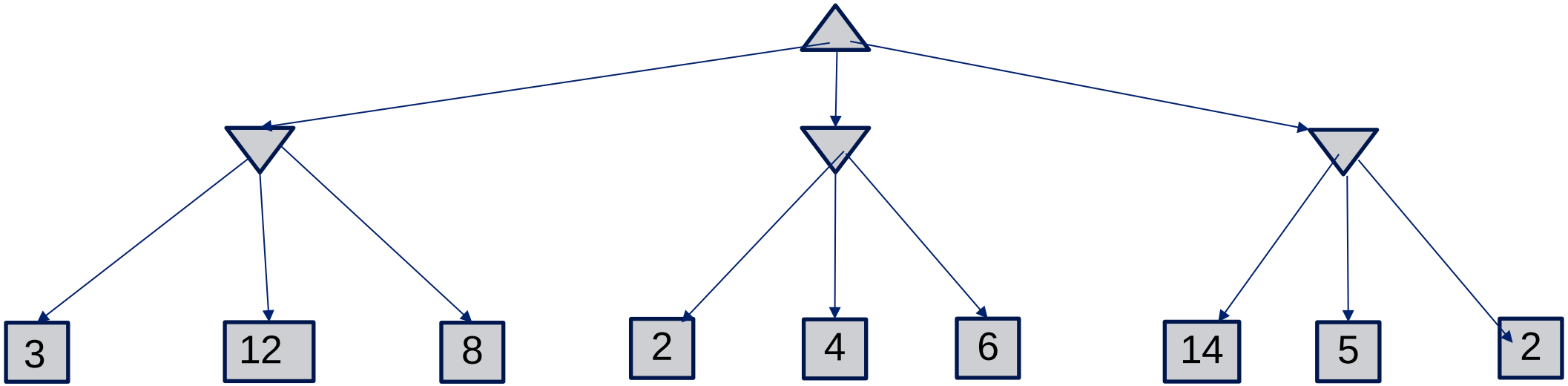
```
def value(state):
    if the state is a terminal state: return the state's utility
    if the next agent is MAX: return max-value(state)
    if the next agent is MIN: return min-value(state)
```



```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor))
    return v
```

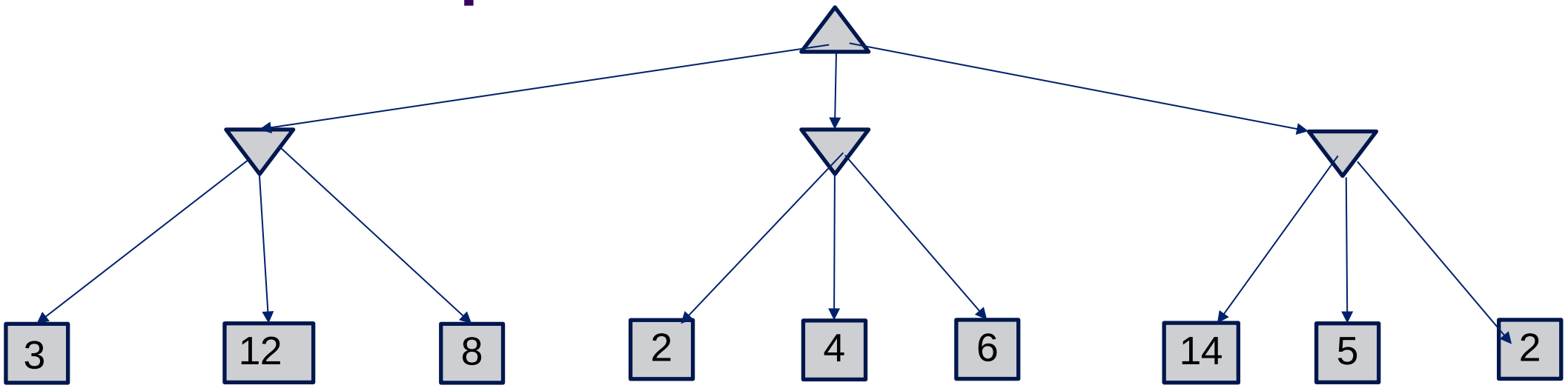
Minimax Example




```
def max-value(state):
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```

```
def min-value(state):
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

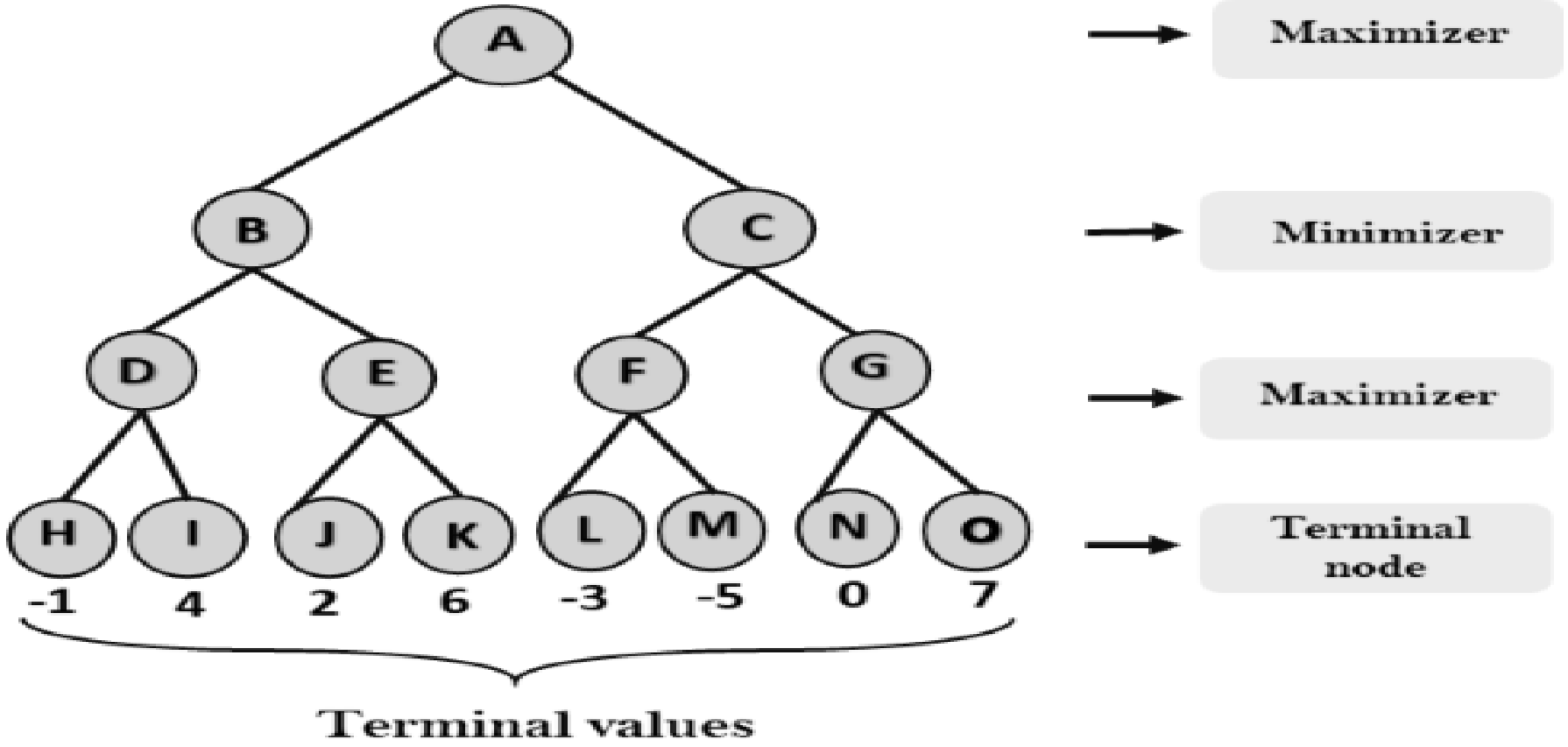
Minimax Example




```
def max-value(state):
    if the state is a terminal state:
        return the state's utility
    initialize v = -∞
    for each successor of state:
        v = max(v, min-value(successor))
    return v
```

```
def min-value(state):
    if the state is a terminal state:
        return the state's utility
    initialize v = +∞
    for each successor of state:
        v = min(v, max-value(successor))
    return v
```

Min-Max Algorithm E.g.....

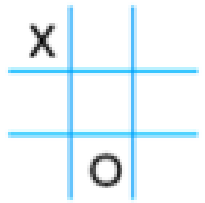


Tic-Tac-Toe Evaluation Function

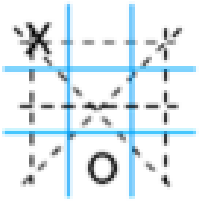
Heuristic: E(n) = M(n) – O(n)

M(n) = Total possible wining lines for MAX

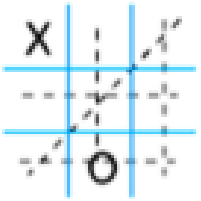
O(n) = Total possible wining lines for the opponent



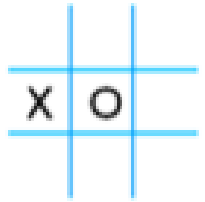
X has 6 possible win paths:



O has 5 possible wins:

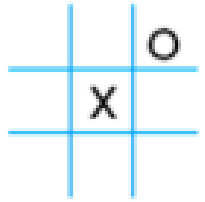


E(n) = 6 – 5 = 1



X has 4 possible win paths;
O has 6 possible wins

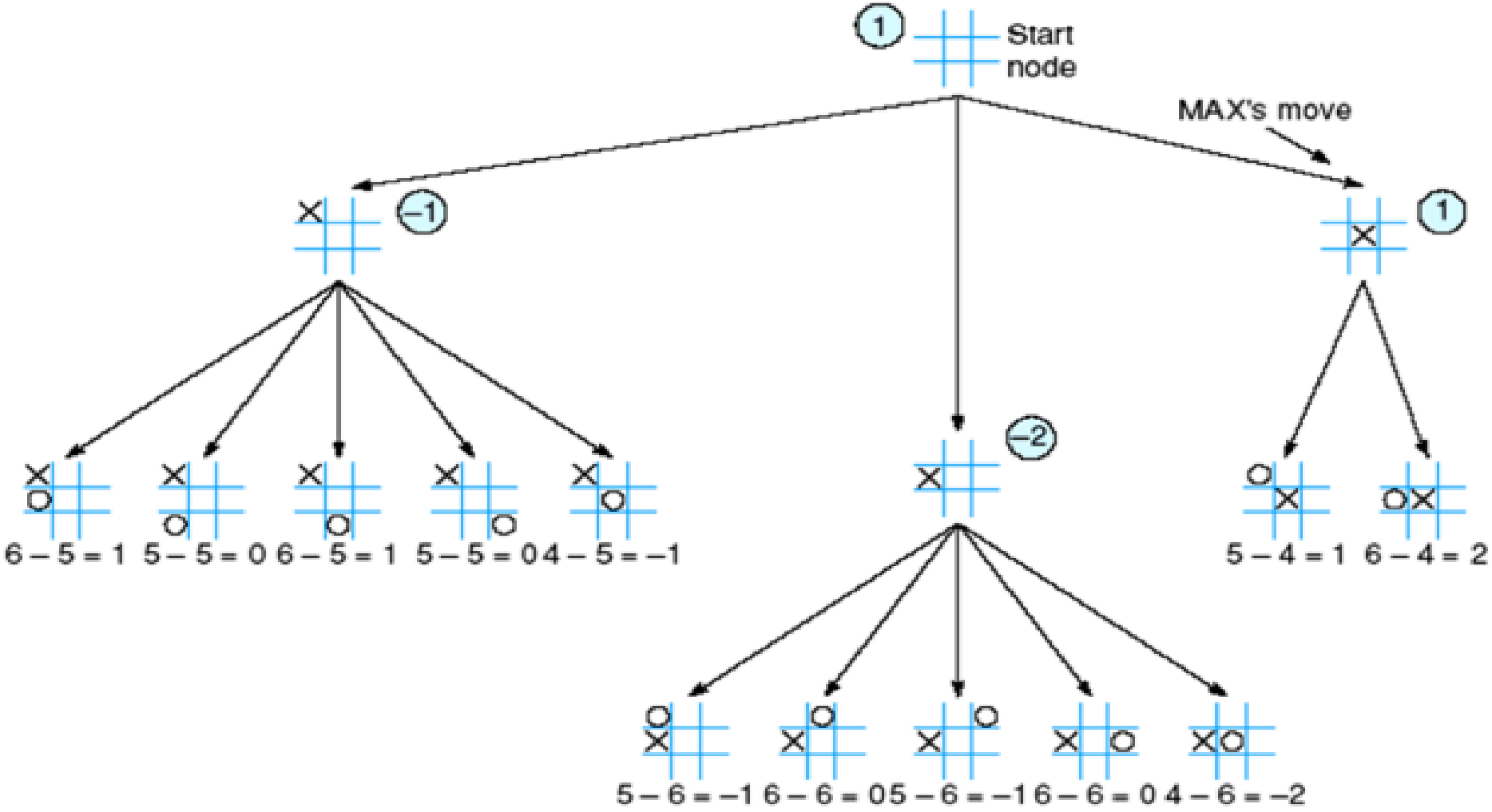
E(n) = 4 – 6 = –2



X has 5 possible win paths;
O has 4 possible wins

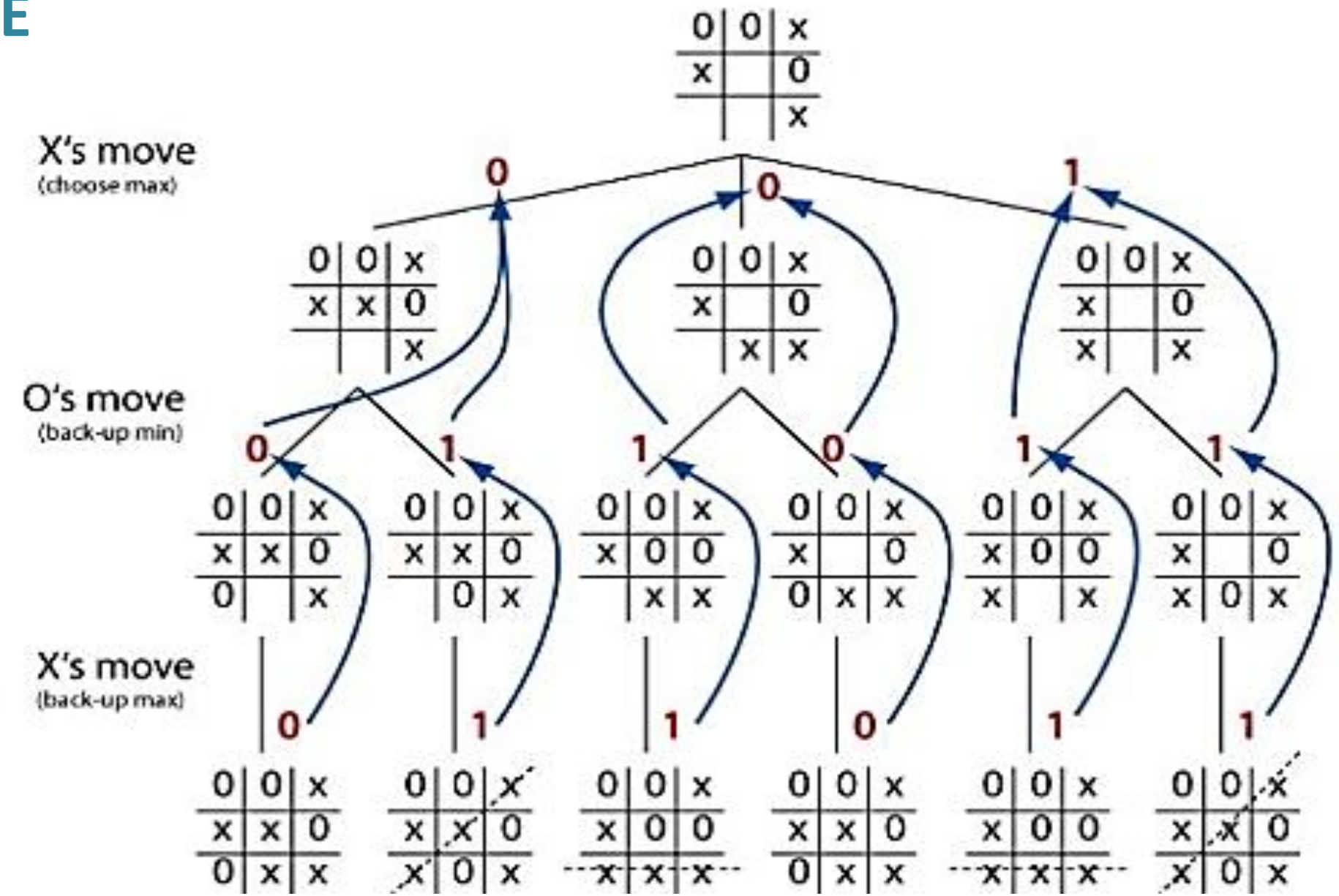
E(n) = 5 – 4 = 1

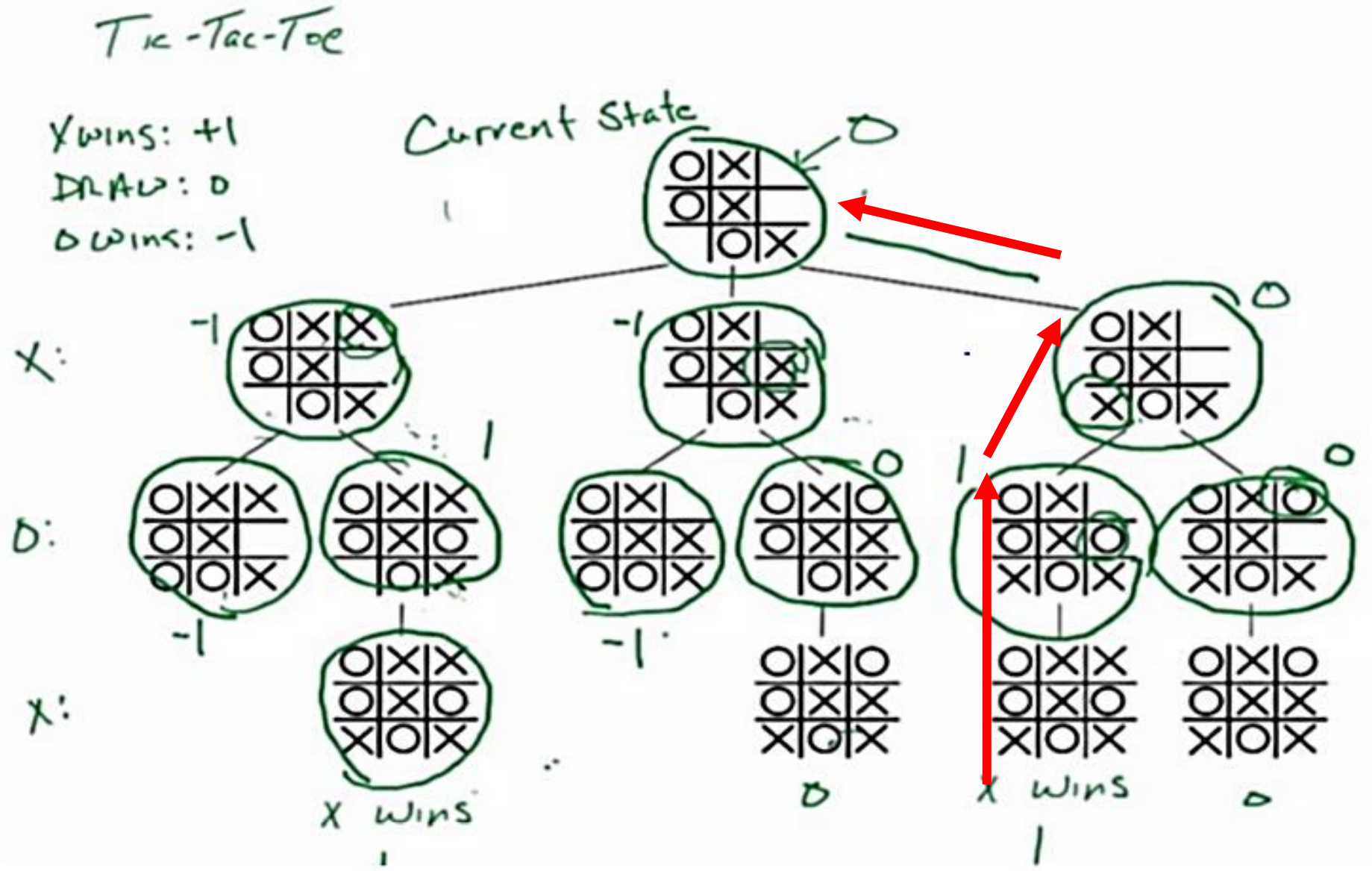
Tic-Tac-Toe Tree at horizon 2 shows the backed up Minimax values



TIC-TAC TOE

E.g.....




Tic-Tac-Toe
X wins: +1
Draw: 0
O wins: -1
Current State
X:
O:
X:
X wins
X wins

Choosing an Action: Minimax Algorithm

- In general, the branching factor and the depth of terminal states are too large to be able to enumerate all possible actions
  - Chess:
    - **Number of states: ~10^{40}**
    - **Branching factor: ~35**
    - **Number of total moves in a game: ~100**
- Solutions
  - Using the current state as the initial state, build the game tree uniformly to the maximal depth ***h*** (called horizon) feasible within the time limit
  - Evaluate the states of the leaf nodes (this requires an evaluation function the estimate the utilities of these nodes)
  - *Back up* the results from the leaves to the root and pick the best action assuming the best play by MIN (and worst for MAX)

Minimax for limited depth

- Create start node as a MAX node with current board configuration
- Expand nodes down to some **depth** (a.k.a. **ply**) of look-ahead in the game
- Apply the evaluation function at each of the leaf nodes
- "Back up" values for each of the non-leaf nodes until a value is computed for the root node
  - At MIN nodes, the backed-up value is the **minimum** of the values associated with its children.
  - At MAX nodes, the backed-up value is the **maximum** of the values associated with its children.
- Pick the move associated with the child node whose backed-up value determined the value at the root

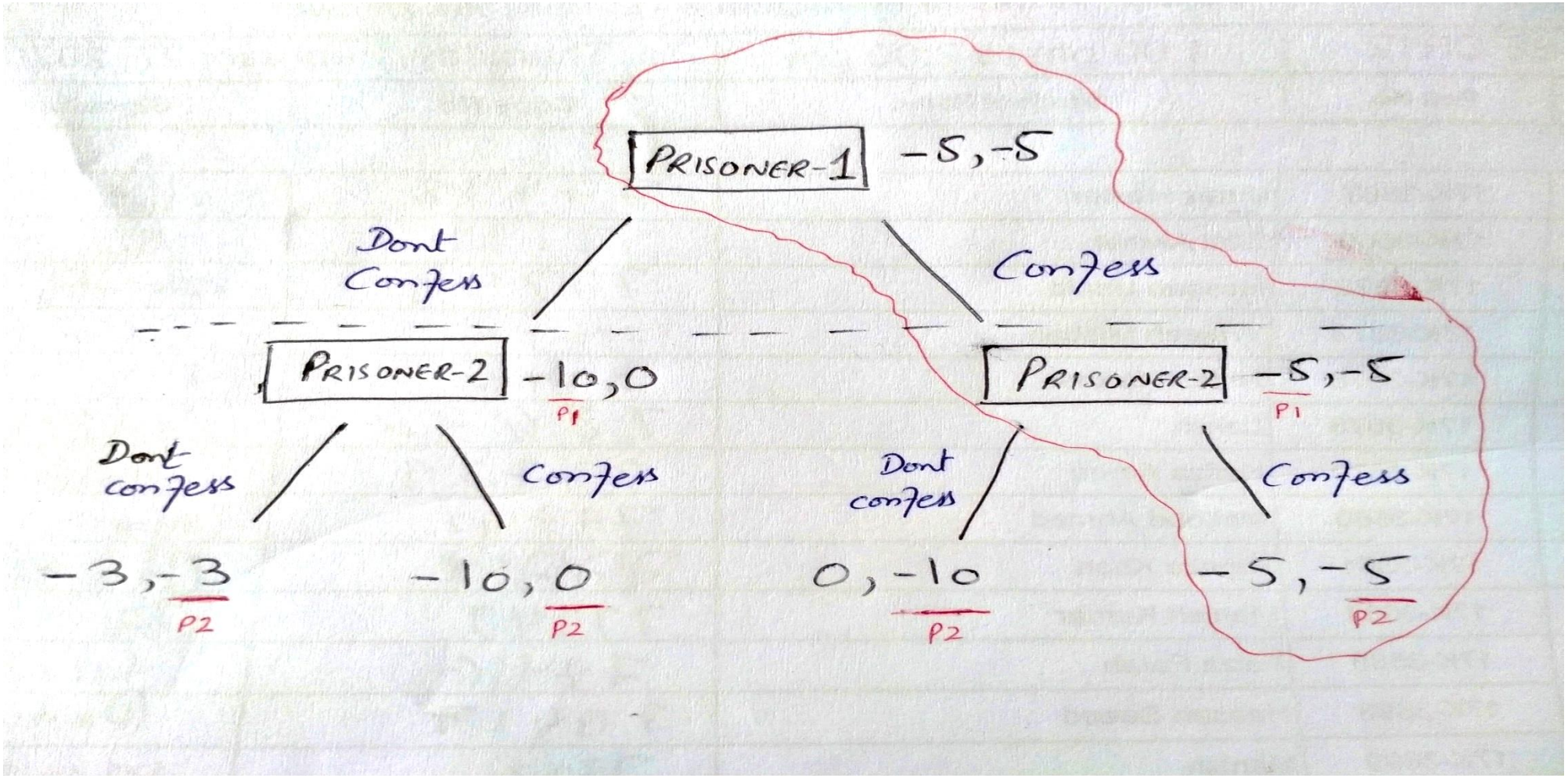
Prisoner's dilemma

- Two criminals have been arrested and the police visit them separately
- If one player testifies against the other and the other refuses, the one who testified goes free and the one who refused gets a 10-year sentence
- If both players testify against each other, they each get a 5-year sentence
- If both refuse to testify, they each get a **3**-year sentence

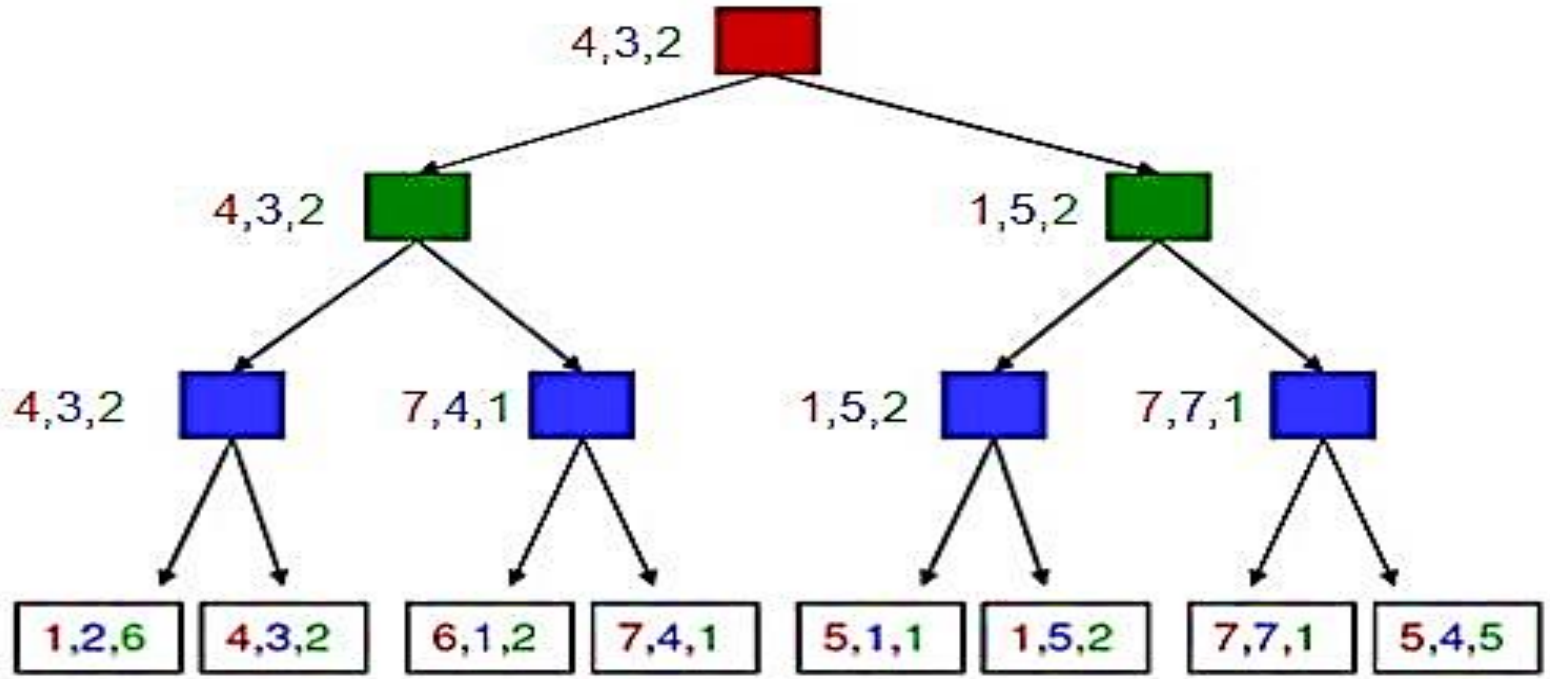
Prisoners' Dilemma: payoff matrix

1 \ 2	Confess	Don't Confess
Confess	-5, -5	0, -10
Don't Confess	-10, 0	-3, -3

Prisoner's Dilemma in Extensive Form



Multi-player, non-zero-sum games



- Utilities are tuples
- Each player maximizes their own utility at each node
- Utilities get propagated (*backed up*) from children to parents

Properties of Mini-Max algorithm:

- **Complete- Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.**
- **Optimal- Min-Max algorithm is optimal if both opponents are playing optimally.**
- **Time complexity- As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^d)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.**
- **Space Complexity- Space complexity of Mini-max algorithm is also similar to DFS which is $O(b^d)$,**

What if MIN does not play optimally?

- **Definition of optimal play for MAX assumes MIN plays optimally:**
  Maximizes worst-case outcome for MAX.
  (Classic game theoretic strategy)

- **But if MIN does not play optimally, MAX will do even better.**
  This theorem is not hard to prove

Comments on Minimax Search

- **Depth-first search with fixed number of ply *m* as the limit.**
  - $O(b^m)$ time complexity – *As usual!*
  - $O(bm)$ space complexity

- **Performance will depend on**
  - the quality of the static evaluation function (expert knowledge)
  - depth of search (computing power and search algorithm)

- **Differences from normal state space search**
  - Looking to make *one* move only, despite deeper search
  - No cost on arcs – costs from backed-up static evaluation
  - MAX can't be sure how MIN will respond to his moves

- **Minimax forms the basis for other game tree search algorithms.**

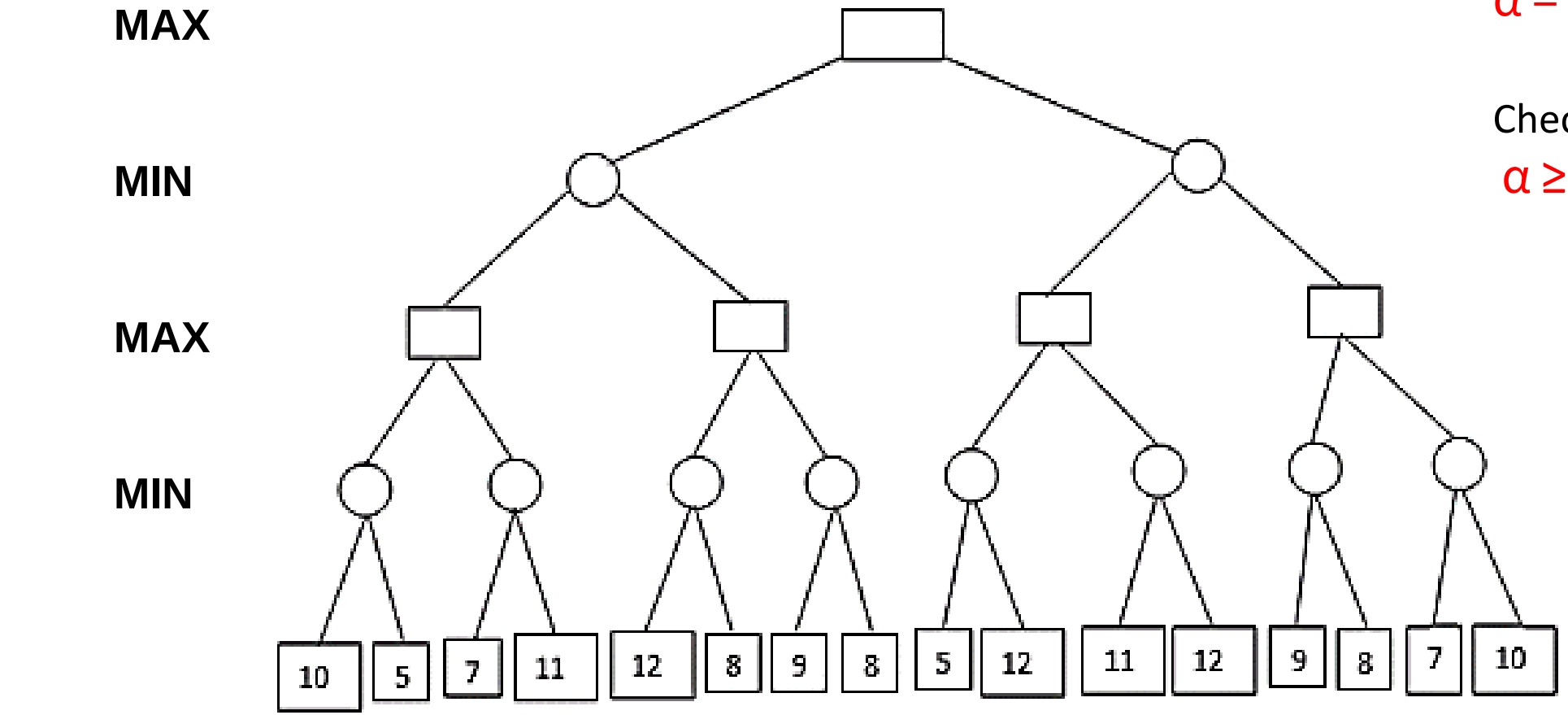
Alpha-Beta Pruning (AIMA 5.3)

Alpha Beta Pruning

It is an optimization technique for the min-max algorithm.
In minmax search, algorithm has to traverse all of the possible game states in depth of the tree i.e. O (b^d) to make decision.
α-β cuts off branches in the game tree which need not be searched because there already exists a better move available.
The two-parameter can be defined as:
α= the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.
β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

Alpha Beta Pruning example:

Considering the first node as max.



At each node initially

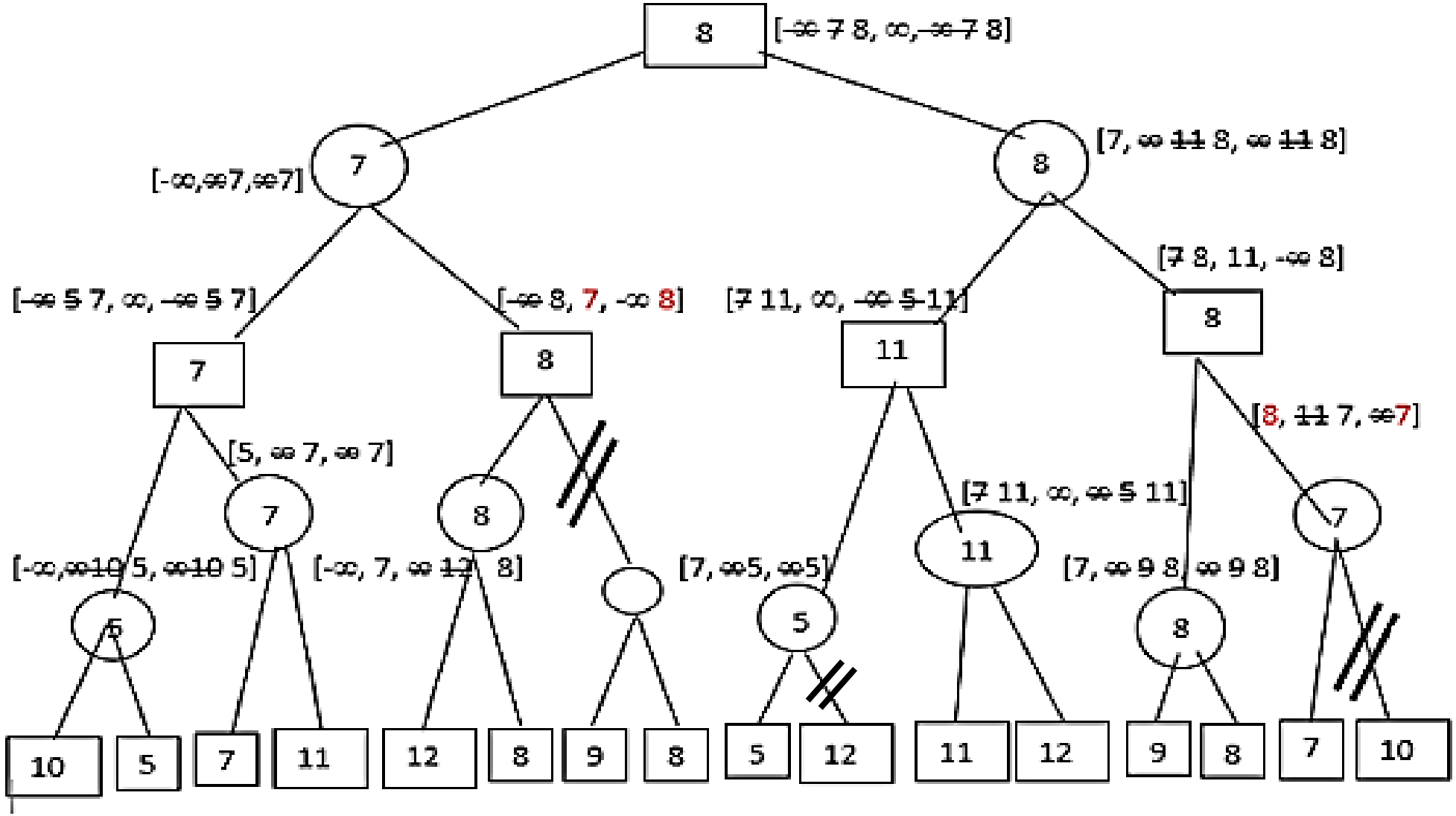
$\alpha = -\infty \quad \beta = \infty$

Check at each node

$\alpha \geq \beta$ (Cut-off)

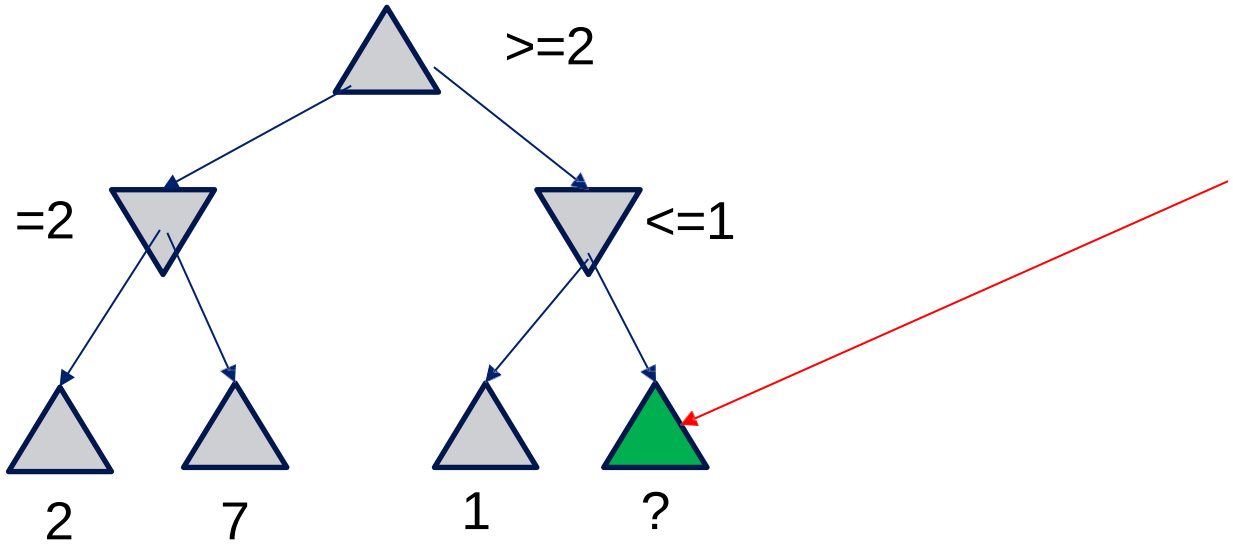
For MAX Nodes, V=max(α) & α >= -∞

For MIN Nodes, V=min(β) & β >= ∞



Alpha-Beta Pruning

- **A way to improve the performance of the Minimax Procedure**
- **Basic idea: *"If you have an idea which is surely bad, don't take the time to see how truly awful it is"* ~ Pat Winston**



- We don't need to compute the valueat this node.

- No matter what this is, it won't changethe value of the root node.

Alpha-Beta Pruning

- **During Minimax, keep track of two additional values:**
  α: MAX's current *lower* bound on MAX's outcome β:
  MIN's current *upper* bound on MIN's outcome

- **MAX will never allow a move that could lead to a worse score (for MAX) than α**
- **MIN will never allow a move that could lead to a better score (for MAX) than β**

- **Therefore, stop evaluating abranch whenever:**
  When evaluating a MAX node: a value $v \geq \beta$ is backed-up
  - MIN will never select that MAX node

  When evaluating a MIN node: a value $v \leq \alpha$ is found
  - MAX will never select that MIN node

Alpha-Beta Implementation

α MAX's best option on path to root
β MIN's best option on path to root

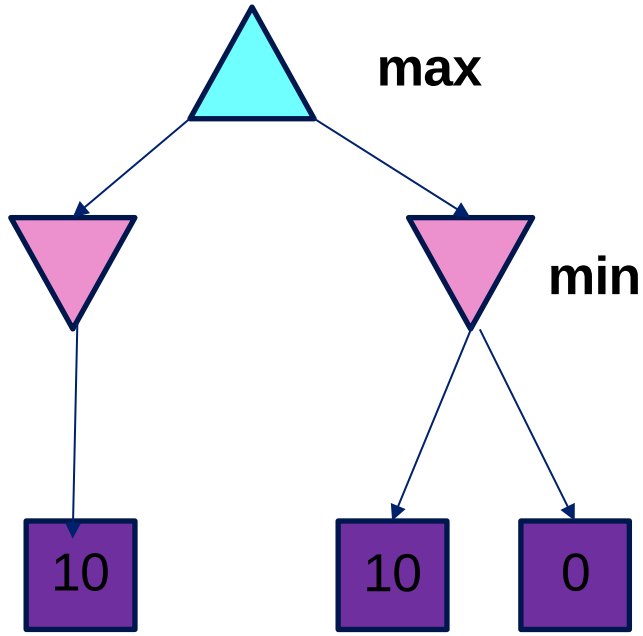
```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

```
def min-value(state , α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```

Alpha-Beta Pruning Properties

- **This pruning has no effect on minimax value computed for the root!**

- **Values of intermediate nodes might be wrong**
  - Important: children of the root may have the wrong value
  - So the most naïve version won't let you do action selection



- **Good child ordering improves effectiveness of pruning**

- **With "perfect ordering":**
  - Time complexity drops to $O(b^{m/2})$
  - Doubles solvable depth!
  - Full search of, e.g. chess, is still hopeless…

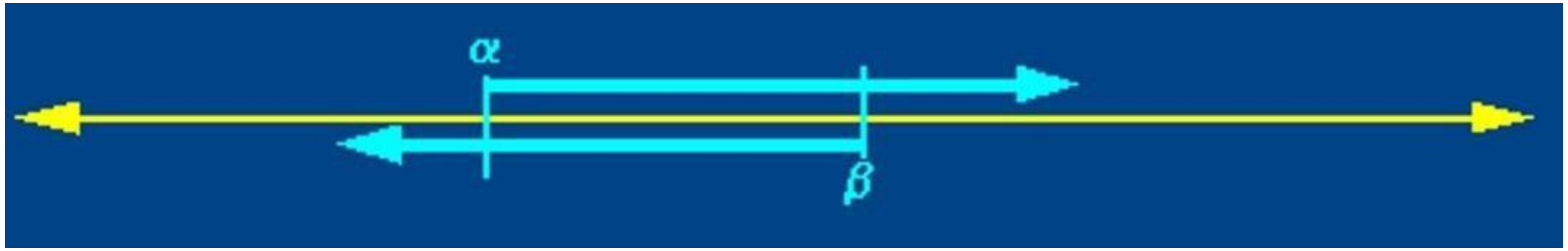
- **This is a simple example of metareasoning (computing about what to compute)**

Alpha-Beta Pruning

- Based on observation that for all viable paths utility value f(*n)* will be α <= f(n) <= β

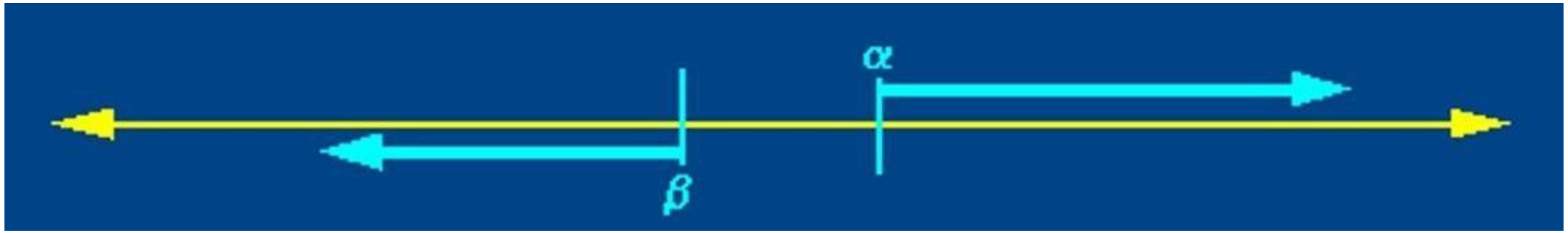
- Initially, α = -infinity, β=infinity

- As the search tree is traversed, the possible utility value window shrinks as α increases, β decreases



Alpha-Beta Pruning

- Whenever the current ranges of alpha and beta no longer overlap, it is clear that the current node is a dead end



Alpha-Beta Pruning (AIMA 5.3)

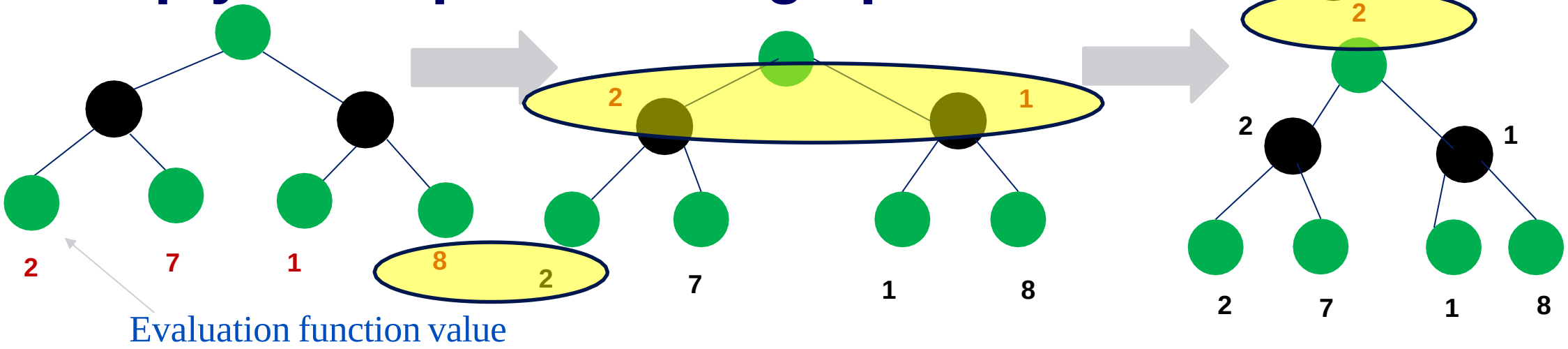
Some slides adapted from Richard Lathrop, USC/ISI, CS 271

Review: The Minimax Rule

- *Idea*: Make the best move for MAX *assuming that MIN always replies with the best move for MIN*

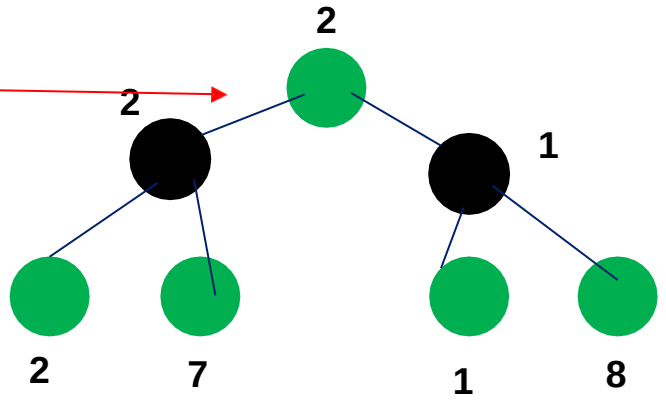
1. **Start with the current position as a MAX node.**
2. **Expand the game tree a fixed number of *ply*.**
3. **Apply the evaluation function to all leaf positions.**
4. **Calculate back-up values bottom-up**:
   - For a **MAX** node, return the ***maximum*** of the values of its children *(i.e. the best for MAX)*
   - For a **MIN** node, return the ***minimum*** of the values of its children *(i.e. the best for MIN*
5. **Pick the move assigned to MAX at the root**
6. **Wait for MIN to respond and REPEAT FROM 1**

2-ply Example: Backing up values



Evaluation function value

This is the move
selected by minimax



New point:
Actually calculated by DFS!

Minimax Algorithm

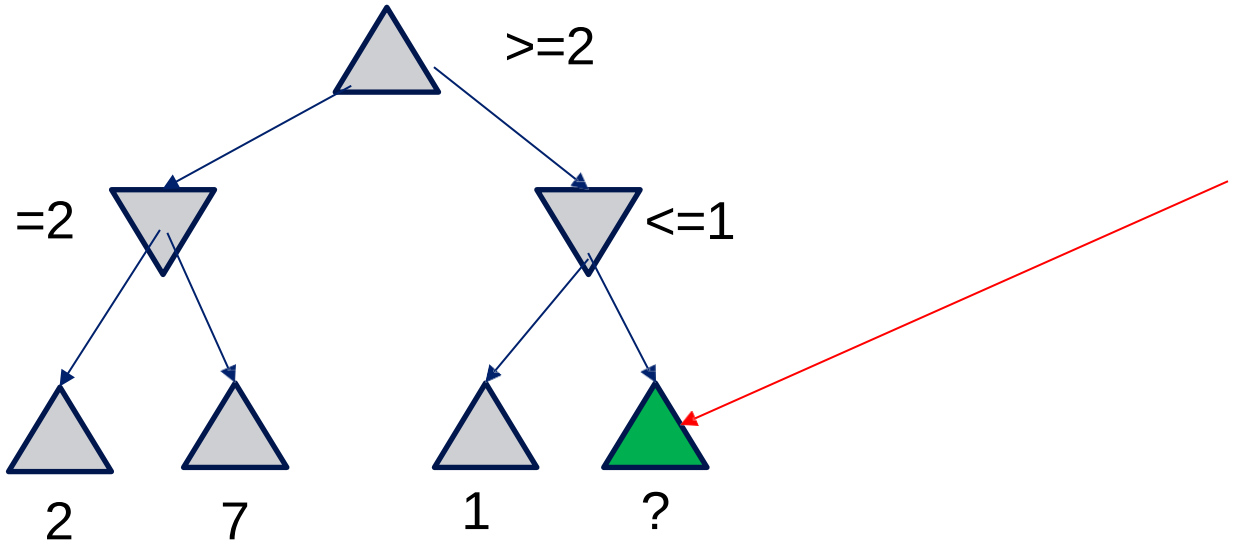
function MINIMAX-DECISION(*state*) **returns** *an action*
  inputs: *state*, current state in game
  v□MAX-VALUE(*state*)
  return an *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  v □-∞
  for *a,s* in SUCCESSORS(*state*) **do**
    v □MAX(*v, MIN-VALUE(s)*)
  return *v*

function MIN-VALUE(*state*) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  v □ ∞
  for *a,s* in SUCCESSORS(*state*) **do**
    v □ MIN(*v, MAX-VALUE(s)*)

Alpha-Beta Pruning

- **A way to improve the performance of the Minimax Procedure**
- **Basic idea: *"If you have an idea which is surely bad, don't take the time to see how truly awful it is"* ~ Pat Winston**



- Assuming left-to-right tree traversal:
- We don't need to compute the valueat this node.
- No matter what this is, it won't changethe value of the root node.

Alpha-Beta Pruning II

- **During Minimax, keep track of two additional values:**
  - **α: current *lower* bound on MAX's outcome**
  - **β: current *upper* bound on MIN's outcome**

- **MAX will never choose a move that could lead to a worse score (for MAX) than α**
- **MIN will never choose a move that could lead to a better score (for MAX) than β**

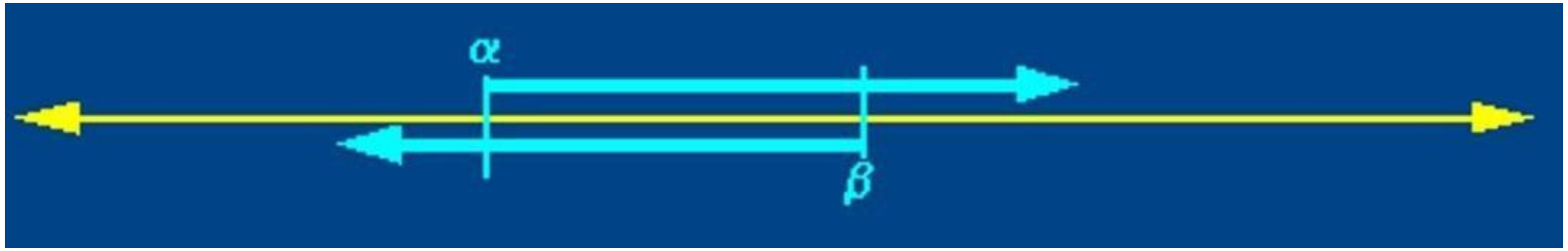
- **Therefore, stop evaluating a branch whenever:**
  - When evaluating a MAX node: a value $v \geq \beta$ is backed-up
    - MIN will never select that MAX node
  - When evaluating a MIN node: a value $v \leq \alpha$ is found
    - MAX will never select that MIN node

Alpha-Beta Pruning IIIa

- Based on observation that for all viable paths utility value f(*n)* will be $\alpha <= f(n) <= \beta$

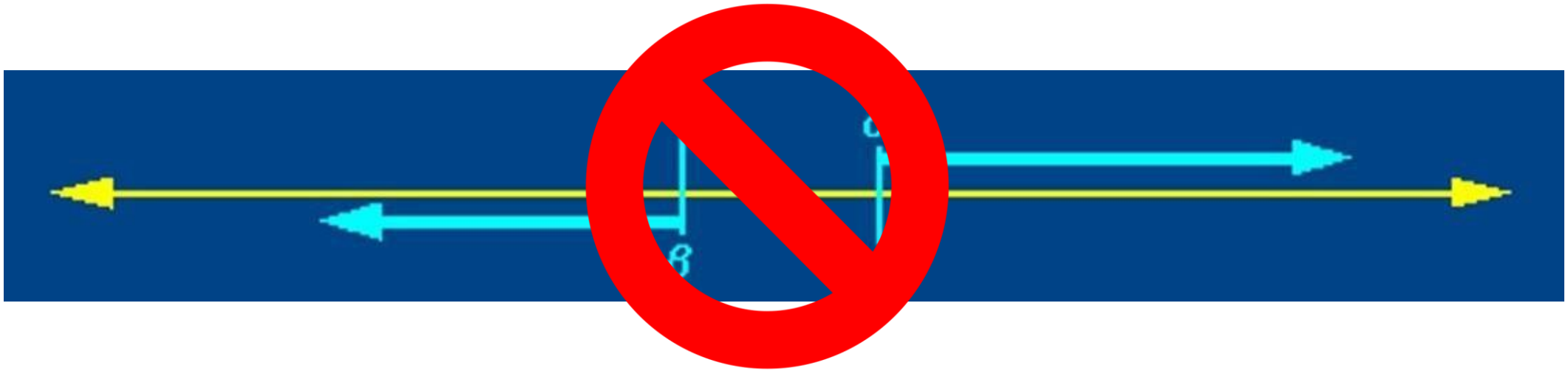
- Initially, α = -infinity, β=infinity

- As the search tree is traversed, the possible utility value window shrinks as α increases, β decreases



Alpha-Beta Pruning IIIb

- Whenever the current ranges of alpha and beta no longer overlap ($\alpha \geq \beta$), it is clear that the current node is a dead end, so it can be pruned



Alpha-beta Algorithm: In detail

- **Depth first search (usually bounded, with static evaluation)**
  only considers nodes along a single path from root at any time

β

- **α = current *lower* bound on MAX's outcome**
  (initially, α = −infinity)
- **β = current *upper* bound on MIN's outcome**
- (initially, β = +infinity)

a

- **Pass current values of a and b *down* to child nodes during search.**
- **Update values of a and b during search:**
  MAX updates α at MAXnodes
  MIN updates β at MIN nodes
- **Prune remaining branches at a node whenever α ≥ β**

When to Prune

- **Prune whenever α ≥ β.**

Prune below a Max node when its α value becomes ≥ the β value of its ancestors.

- **Max nodes update α** based on children's returned values.
- Idea: Player MIN at node above won't pick that value anyway, since MIN can force a worse value.

Prune below a Min node when its β value becomes ≤ the α value of its ancestors.

- **Min nodes update β** based on children's returned values.
- Idea: Player MAX at node above won't pick that value anyway; she can do better.

Pseudocode for Alpha-Beta Algorithm

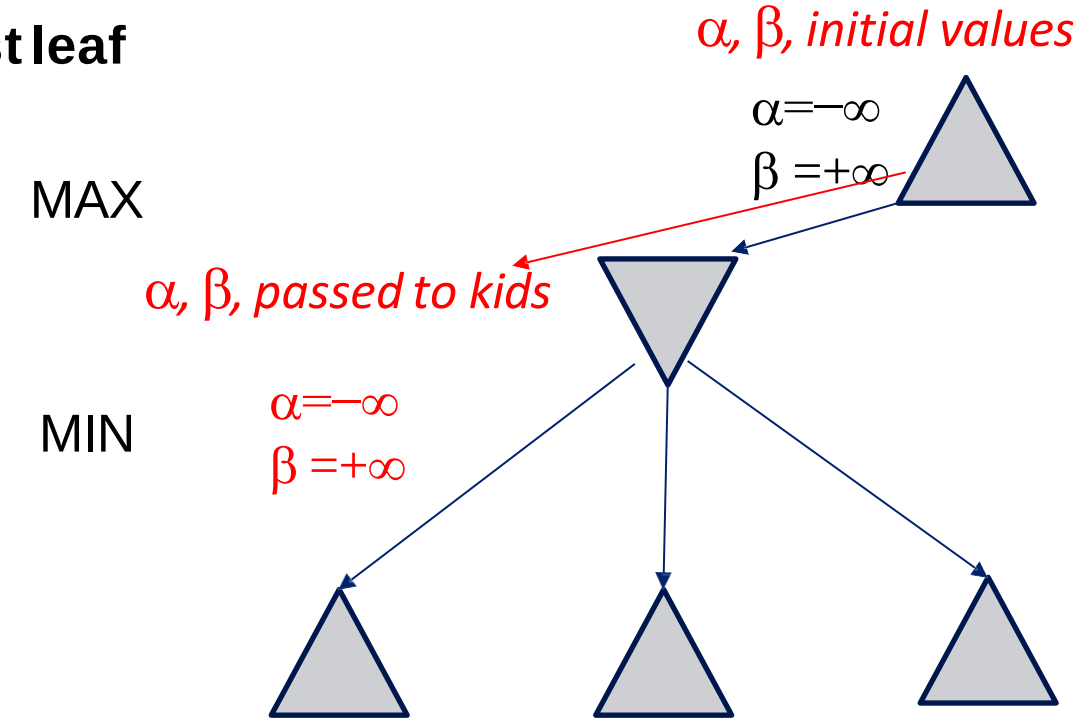
```
function ALPHA-BETA-SEARCH(state) returns an action
    inputs: state, current state in game
    v□MAX-VALUE(state, - ∞ , +∞)
    return an action in ACTIONS(state) with value v
```

An Alpha-Beta Example

Do DF-search until first leaf

β

α



An Alpha-Beta Example (continued)

B

↓

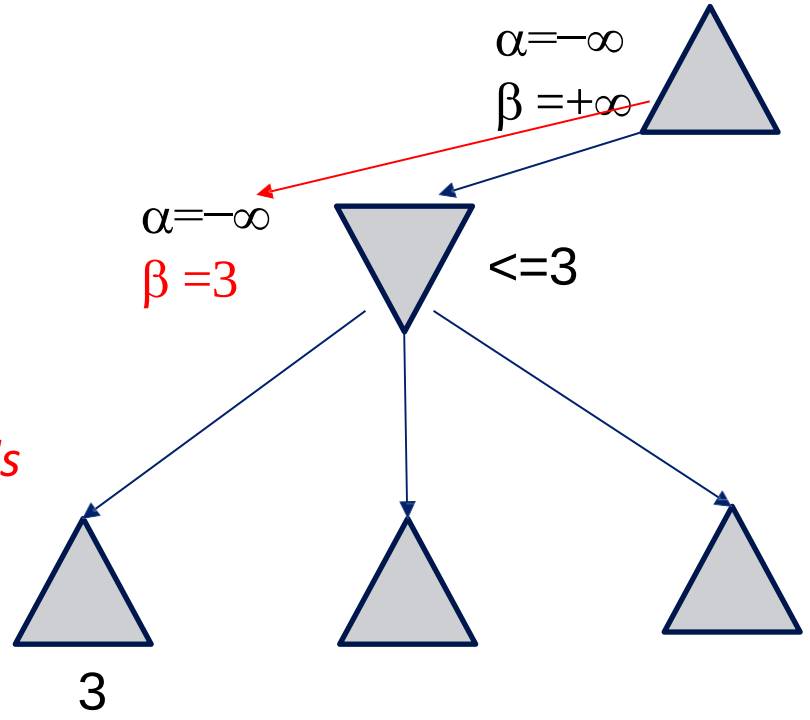
MAX

MIN

↑

a

MIN updates β, based on kids



An Alpha-Beta Example (continued)

β

↓

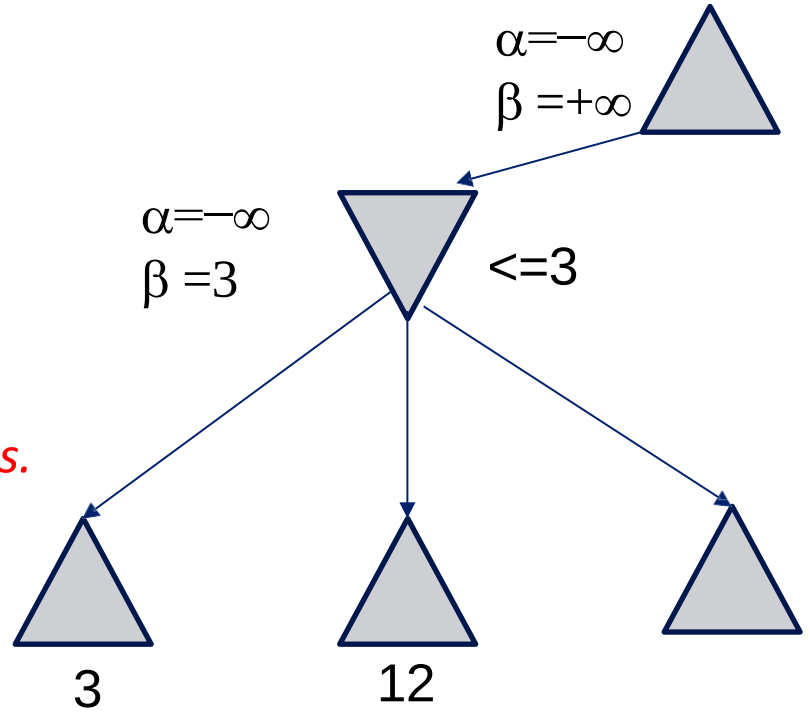
MAX

MIN

↑

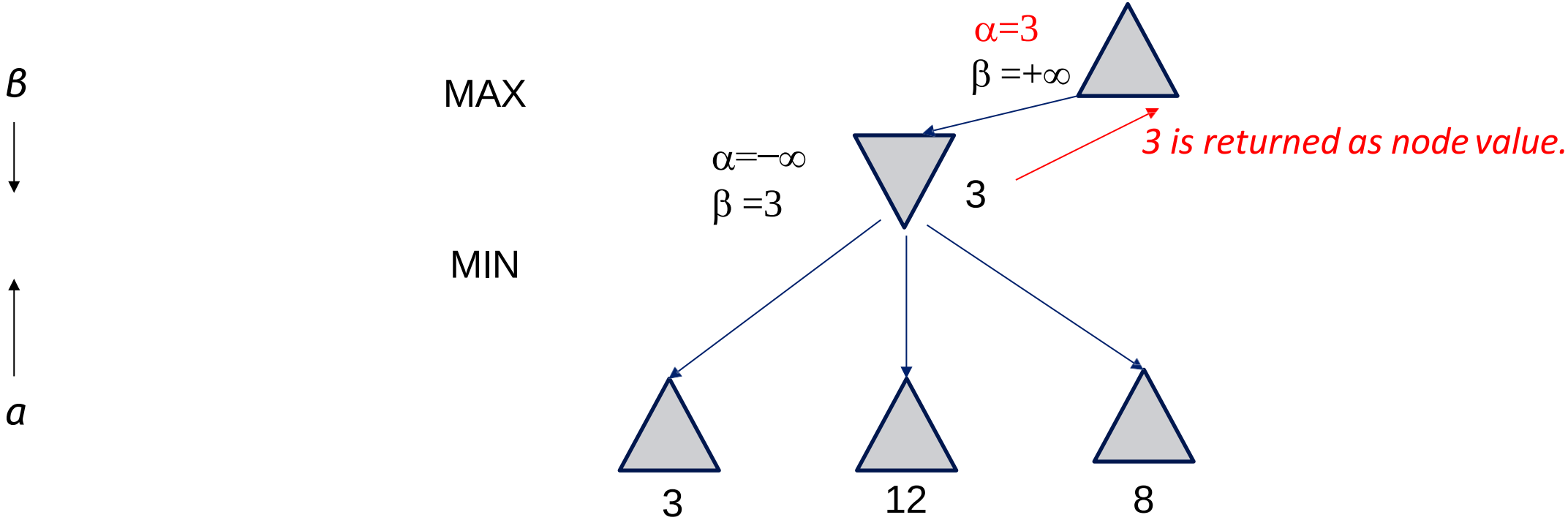
α

MIN updates β, based on kids.
No change.

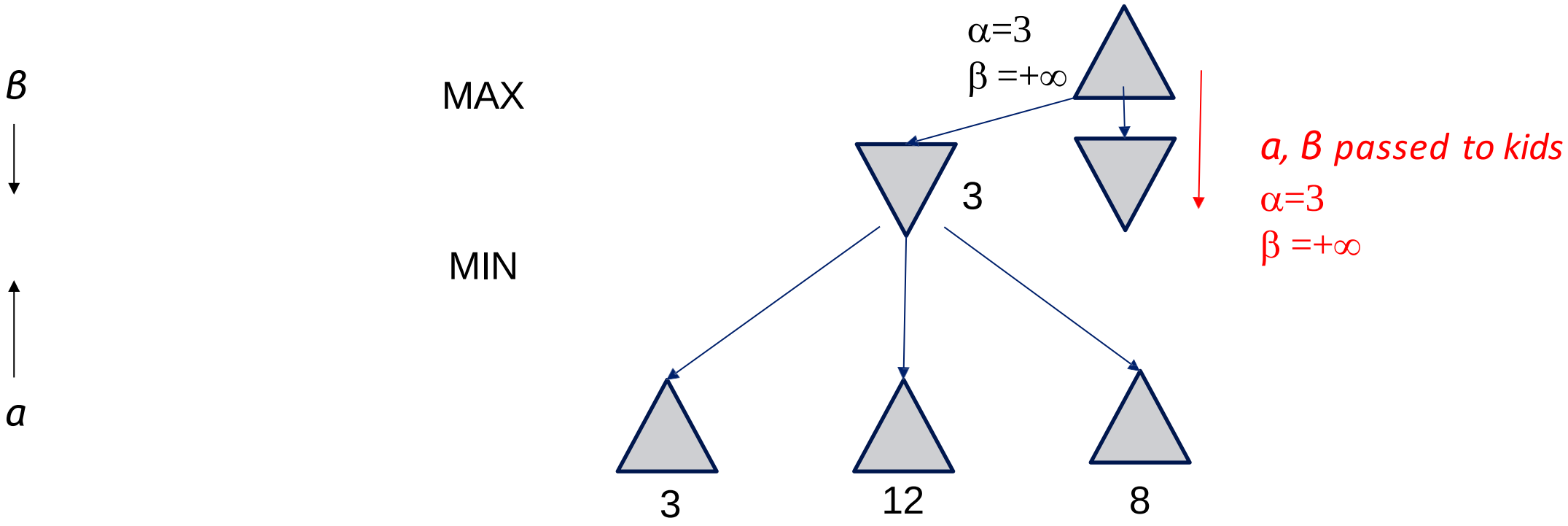


An Alpha-Beta Example (continued)

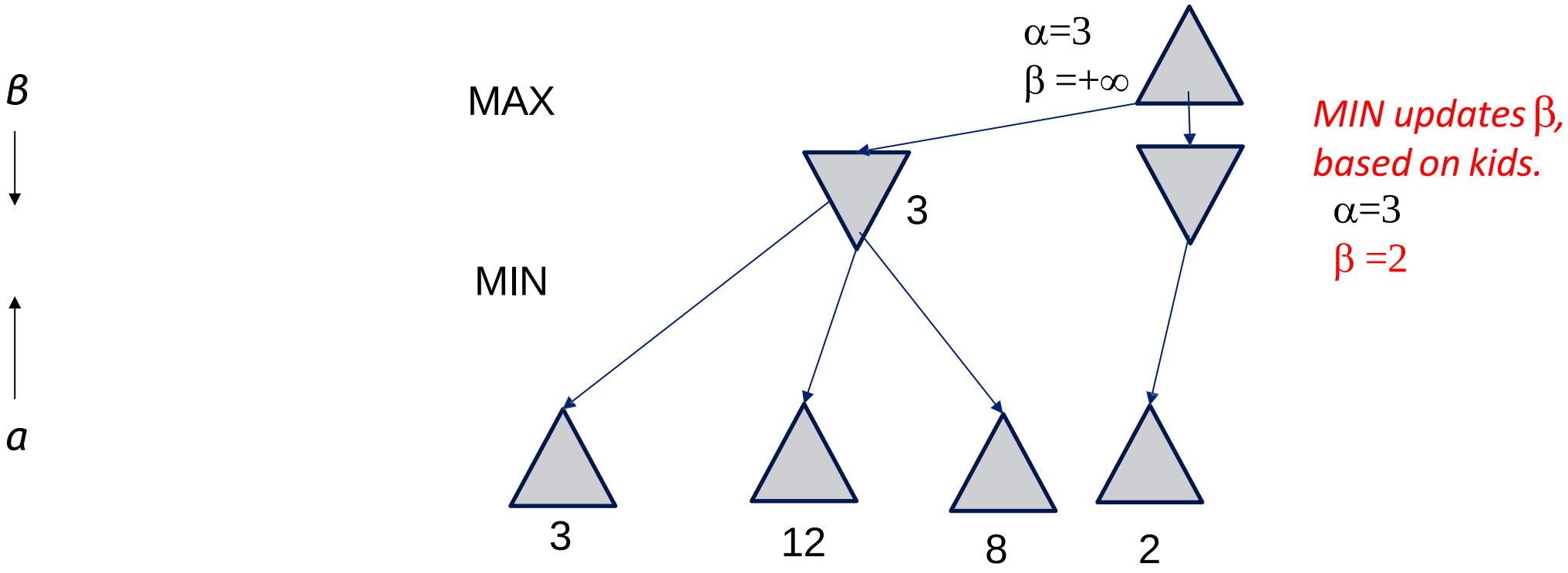
MAX updates , based a on kids.



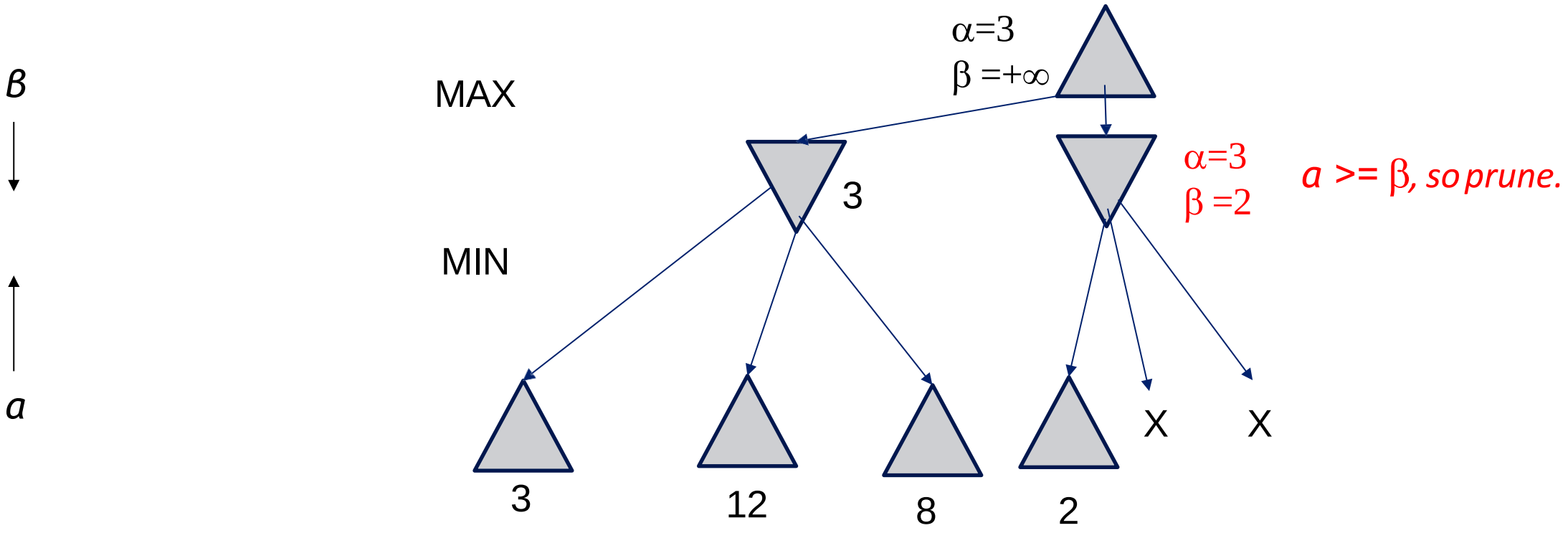
An Alpha-Beta Example (continued)



An Alpha-Beta Example (continued)

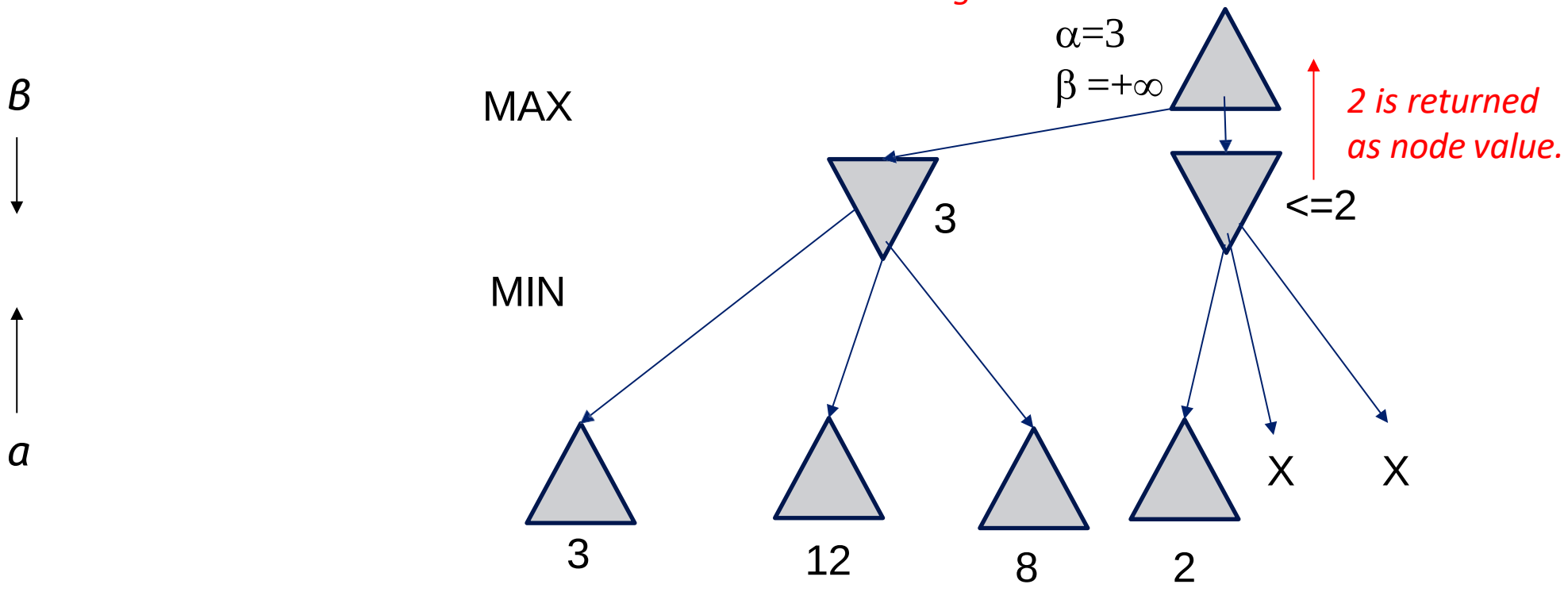


An Alpha-Beta Example (continued)

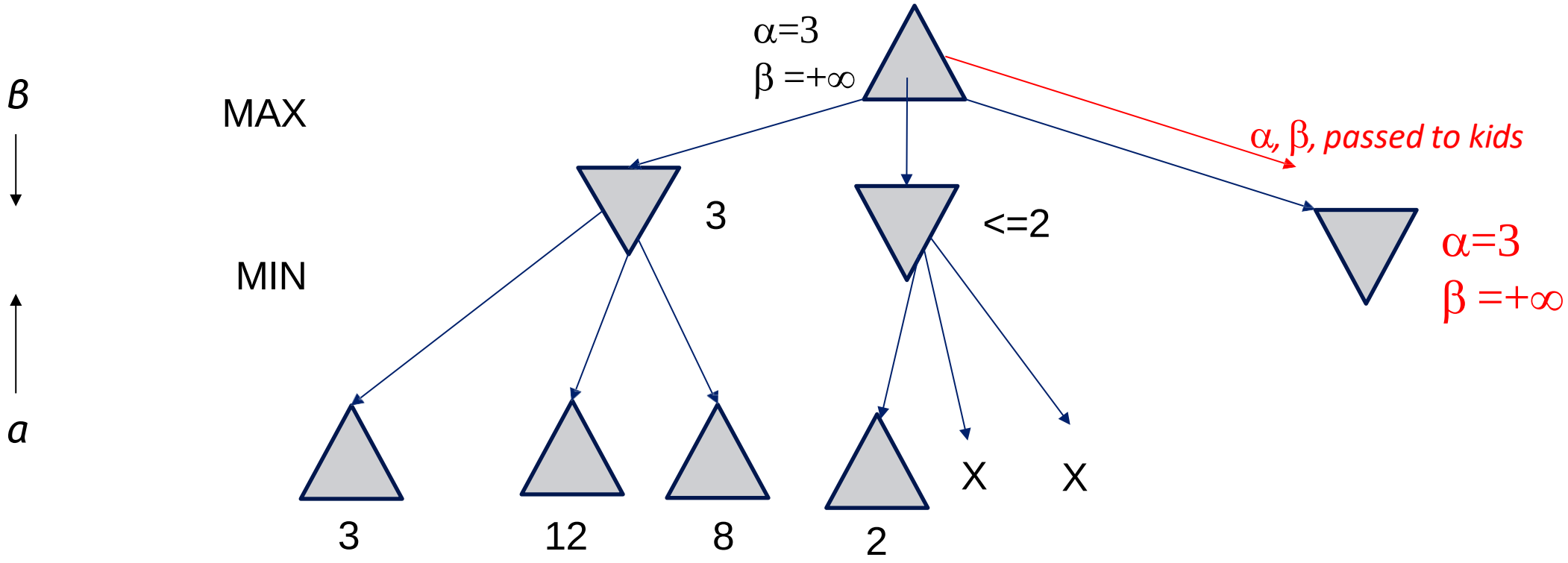


An Alpha-Beta Example (continued)

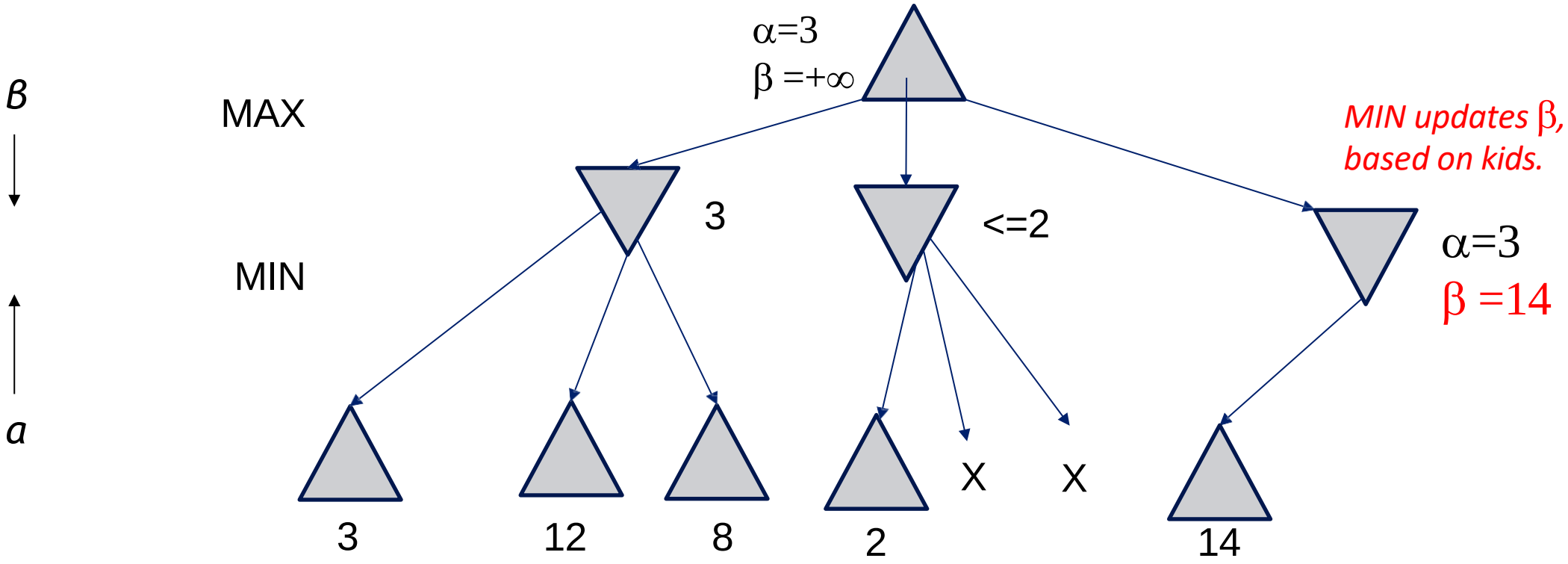
MAX updates α, based on kids.
No change.



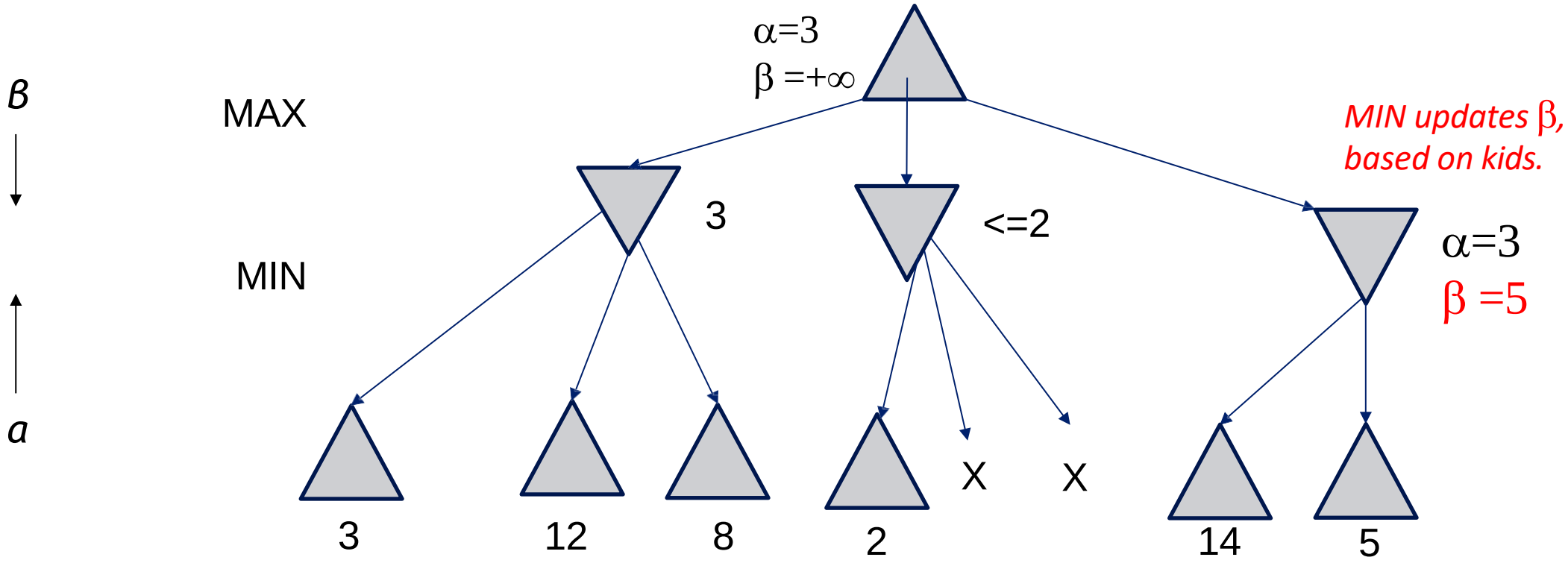
An Alpha-Beta Example (continued)



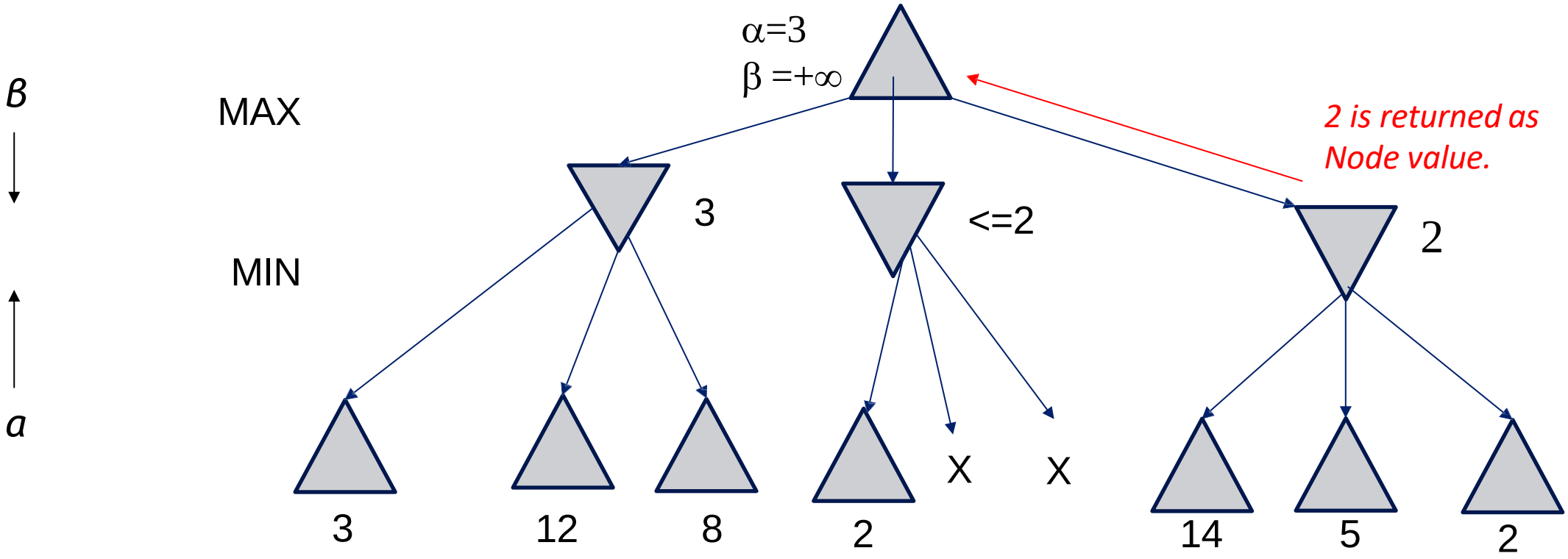
An Alpha-Beta Example (continued)



An Alpha-Beta Example (continued)

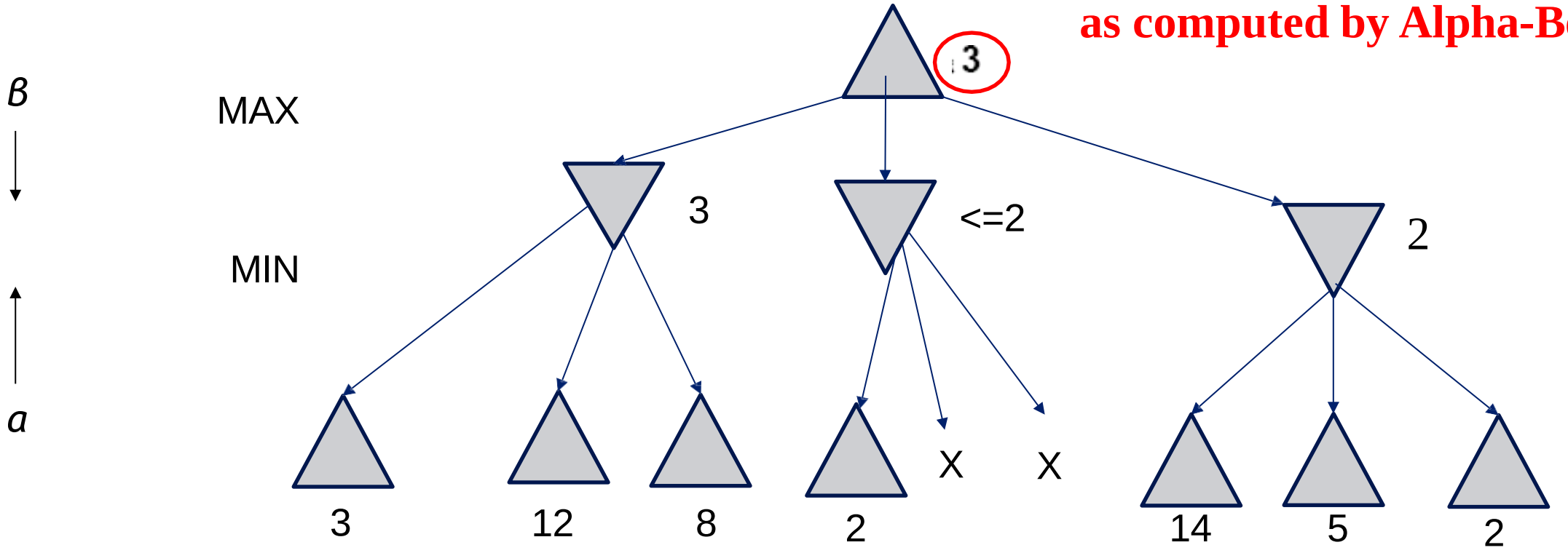


An Alpha-Beta Example (continued)



An Alpha-Beta Example (continued)

Max now makes it's best move as computed by Alpha-Beta



Alpha-Beta Algorithm Pseudocode

```
function ALPHA-BETA-SEARCH(state) returns an action
    inputs: state, current state in game
    v□MAX-VALUE(state, - ∞ , +∞)
    return an action in ACTIONS(state) with value v
```

```
function MAX-VALUE(state,□ , □) returns a utility value
   if TERMINAL-TEST(state) then return UTILITY(state)
   v □ - ∞
   for a in ACTIONS(state) do
        v □ MAX(v,MIN-VALUE(Result(s,a), □ , □))
        if v ≥ □ then return v
        □ □ MAX(□ ,v)
   return v
```

Alpha-Beta Algorithm II

```
function MIN-VALUE(state, □ , □) returns a utility value
   if TERMINAL-TEST(state) then return UTILITY(state)
   v □ + ∞
   for a,s in SUCCESSORS(state) do
       v □ MIN(v,MAX-VALUE(s, □ , □))
       if v ≤ □ then return v
       □ □ MIN(□ ,v)
   return v
```

Effectiveness of Alpha-Beta Pruning

- **Guaranteed to compute same root value as Minimax**
- **Worst case:** no pruning, same as Minimax ($O(b^d)$)
- **Best case:** when each player's best move is the first option examined, examines only $O(b^{d/2})$ nodes, allowing to search twice as deep!

How much do we gain?

- Alpha-beta is guaranteed to compute the same value for the root node as computed by minimax, with less or equal computation
- Assume a game tree of uniform branching factor b
  - Minimax examines $O(b^m)$ nodes, so does alpha-beta in the worst-case
- The gain for alpha-beta is maximum when:
  - The MIN children of a MAX node are ordered in increasing backed up values
  - The MAX children of a MIN node are ordered in decreasing backed up values
- Then alpha-beta examines $O(b^{m/2})$ nodes [Knuth and Moore, 1975]
  - But this requires an oracle (if we knew how to order nodes perfectly, we would not need to search the game tree)
- If nodes are ordered at random, then the average number of nodes examined by alpha-beta is $\sim O(b^{3m/4})$
- In Deep Blue, they found empirically that alpha-beta pruning meant that the average branching factor at each node was about 6 instead of about 35!

When best move is the firstexamined, examines only $O(b^{d/2})$ nodes….

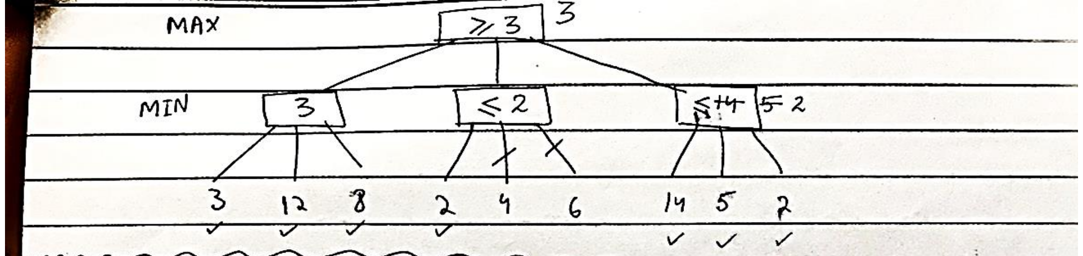
- So: run Iterative Deepening search, sort by value returned on last iteration.
- So: expand captures first, then threats, then forward moves, etc.

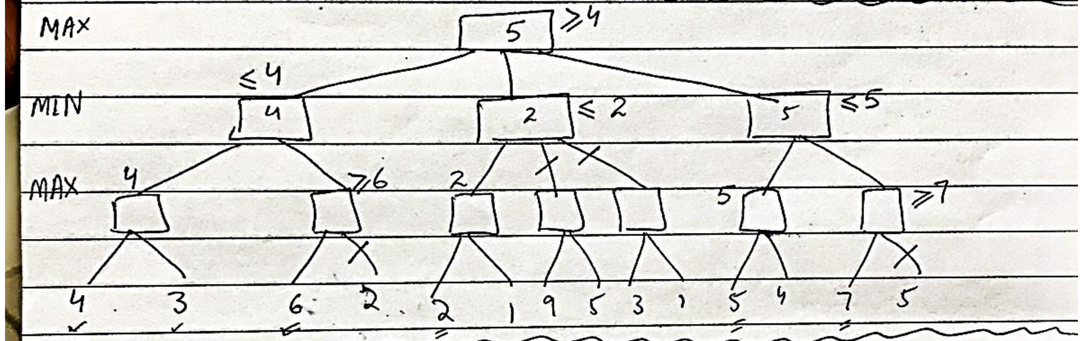
- **$O(b^{(d/2)})$** is the same as having a branching factor of sqrt(b),
  Since $(\text{sqrt}(b))^d = b^{(d/2)}$
  e.g., in chess go from b ~ 35 to b ~ 6

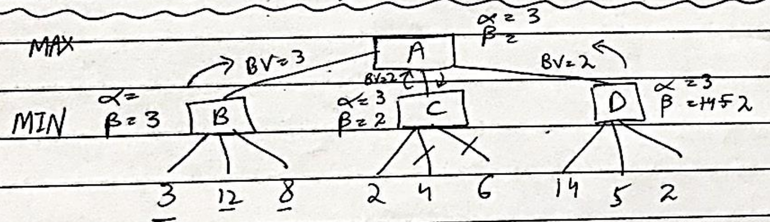
- **For Deep Blue, alpha-beta pruning reduced the average branching factorfrom 35-40 to 6, as expected, doubling search depth**

Real systems use a few more tricks

- **Expand the proposed solution a little farther**
  - Just to make sure there are no surprises
- **Learn better board evaluation functions**
  - E.g., for backgammon
- **Learn model of your opponent**
  - E.g., for poker
- **Do stochastic search**
  - E.g., for go







$\alpha \to MAX \to -\infty$

$\beta \to MIN \to \infty$

BV → best value

if $\beta \le \alpha$ break;

$B \Rightarrow \beta = \min(\infty, 3) = 3$

$\beta = \min(3, 12) = 3$

$\beta = \min(3, 8) = 3$

$A \Rightarrow \alpha = \max(-\infty, 3) = 3$

$C \Rightarrow \beta = \min(\infty, 2) = 2$

$A \Rightarrow \alpha = \max(3, 2) = 3$

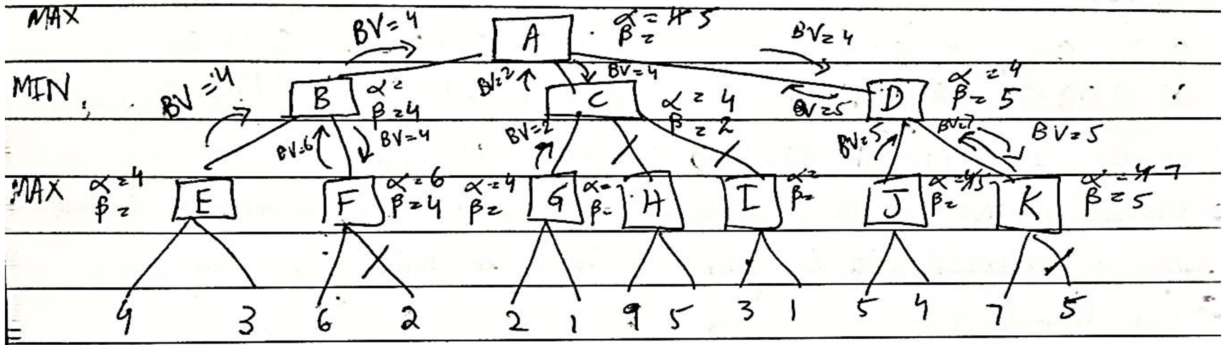
$D \Rightarrow \beta = \min(\infty, 14) = 14$

$\beta = \min(14, 5) = 5$

$\beta = \min(5, 2) = 2$

$A \Rightarrow \alpha = \max(3, 2) = 3$

M.M.



if $\beta \le \alpha$ break;

$E \Rightarrow \alpha = \max(-\infty, 4) = 4$

$\alpha = \max(4, 3) = 4$

$B \Rightarrow \beta = \min(\infty, 4) = 4$

$F \Rightarrow \alpha = \max(-\infty, 6) = 6$

$B \Rightarrow \beta = \min(4, 6) = 4$

$A \Rightarrow \alpha = \max(-\infty, 4) = 4$

$G \Rightarrow \alpha = \max(4, 2) = 4$

$\alpha = \max(4, 1) = 4$

$C \Rightarrow \beta = \min(\infty, 2) = 2$

$A \Rightarrow \alpha = \max(4, 2) = 4$

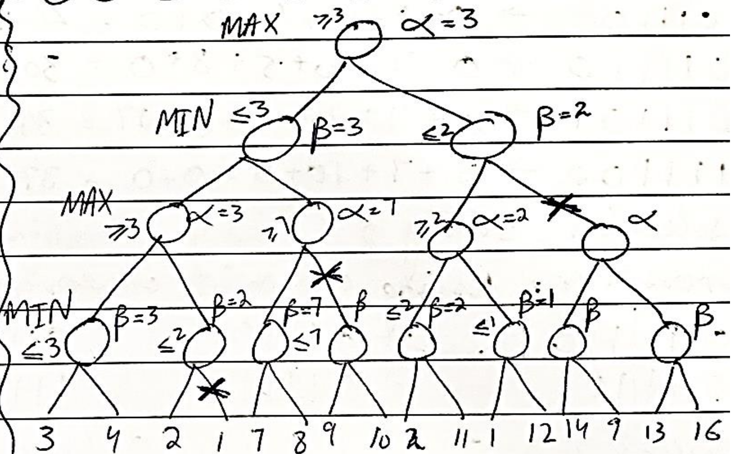
$J \Rightarrow \alpha = \max(4, 5) = 5$

$\alpha = \max(5, 4) = 5$

$D \Rightarrow \beta = \min(\infty, 5) = 5$

$K \Rightarrow \alpha = \max(4, 7) = 7$

$A \Rightarrow \alpha = \max(4, 5) = 5$



Reference Material

- https://www.youtube.com/watch?v=fR5p7vZ2jKU&list=PL_2v8IwL_PfzUgpj2cICVqqTIp6YT4wE6&index=1
- https://www.youtube.com/watch?v=3r3CfvPT0_E&list=PL_2v8IwL_PfzUgpj2cICVqqTIp6YT4wE6&index=2
- https://www.youtube.com/watch?v=rpfEvu7QdHM&list=PL_2v8IwL_PfzUgpj2cICVqqTIp6YT4wE6&index=3
- https://www.youtube.com/watch?v=FFzdXJ49KAI&list=PLxCzCOWd7aiHGhOHV-nwb0HR5US5GFKFI&index=18
- https://www.youtube.com/watch?v=Ntu8nNBL28o&list=PLxCzCOWd7aiHGhOHV-nwb0HR5US5GFKFI&index=19
- https://www.youtube.com/watch?v=dEs_kbvu_0s&list=PLxCzCOWd7aiHGhOHV-nwb0HR5US5GFKFI&index=20